MARRAKECH
جامعة القاضي عياض
UNIVERSITÉ CADI AYYAD

المدرسة العليا
للتكنولوجيا - قلعة السراغنة
ÉCOLE SUPÉRIEURE DE TECHNOLOGIE
D'EL KELÂA DES SRAGHNA




# MACHINE LEARNING

Prof S.ETTARFAOUI

# PLAN DU COURS

1. INTRODUCTION
2. Pourquoi l'Apprentissage Automatique?
3. Définitions et concepts de base
4. Les Bases du Machine Learning
5. Algorithmes de machine learning
6. Les Types d'Apprentissage
   - Apprentissage Supervisé
   - Apprentissage Non Supervisé
   - Apprentissage par Renforcement
7. CONCLUSION

# MACHINE LEARNING
# (APPRENTISSAGE AUTOMATIQUE)

## INTRODUCTION:

Le machine learning, ou apprentissage automatique en français, est une branche de l'intelligence artificielle qui permet aux machines d'apprendre à partir de données, sans être explicitement programmées pour chaque tâche.

### Algorithme vs. Machine Learning : Quelle est la différence ?

Un **algorithme** est une suite d'instructions bien définies et ordonnées qui permettent de résoudre un problème spécifique.

Les algorithmes sont souvent conçus pour des tâches précises et répétitives, et leur comportement est entièrement déterminé par les instructions que vous leur donnez.

# MACHINE LEARNING
# (APPRENTISSAGE AUTOMATIQUE)

## Algorithme vs. Machine Learning : Quelle est la différence ?

Imaginons que vous voulez trier une liste de nombres. Un algorithme de tri (comme le tri à bulles ou le tri rapide) vous permet de définir une série d'étapes pour ordonner cette liste. Le **machine learning** pourrait être utilisé pour prédire la valeur du prochain nombre dans la liste en analysant les nombres précédents.

**Machine Learning**: Un ensemble de méthodes qui permettent à une machine d'apprendre à partir de données.

Les **algorithmes** sont au cœur du **machine learning**. Les algorithmes de machine learning utilisent des techniques mathématiques et statistiques pour trouver des modèles dans les données et faire des prédictions.

# MACHINE LEARNING
# (APPRENTISSAGE AUTOMATIQUE)

## Algorithme vs. Machine Learning : Quelle est la différence ?

**Le machine learning permet de créer des algorithmes plus complexes et plus adaptatifs.** Par exemple, un algorithme de machine learning peut être utilisé pour reconnaître des visages dans des images, tandis qu'un algorithme traditionnel nécessiterait des règles très spécifiques pour chaque visage.

**En conclusion**, le machine learning est une évolution des algorithmes traditionnels. Il permet de créer des systèmes plus intelligents et plus autonomes, capables d'apprendre et de s'adapter à de nouvelles situations.

# MACHINE LEARNING

## Les avantages du machine learning:

**Automatisation des tâches:** Le machine learning permet d'automatiser des tâches répétitives et fastidieuses, libérant ainsi du temps pour des activités à plus forte valeur ajoutée.

**Amélioration de la précision:** Les modèles de machine learning peuvent atteindre des niveaux de précision très élevés, notamment dans des domaines comme la reconnaissance d'images et la traduction automatique.

**Découverte de patterns cachés:** Le machine learning permet de découvrir des patterns et des corrélations dans les données qui seraient difficiles à identifier par l'homme.

**Personnalisation:** Les modèles de machine learning peuvent être utilisés pour créer des expériences utilisateur personnalisées, en fonction des préférences et des comportements de chaque individu.

**Prédictions:** Le machine learning permet de faire des prédictions sur des événements futurs, ce qui est particulièrement utile dans des domaines comme la finance et la santé.

# MACHINE LEARNING

## Les défis du machine learning:

**Qualité des données:** La qualité des données utilisées pour entraîner un modèle de machine learning est cruciale. Des données biaisées ou incomplètes peuvent conduire à des résultats erronés.

**Complexité:** Les modèles de machine learning peuvent être complexes et difficiles à comprendre, ce qui peut rendre leur mise en œuvre et leur maintenance coûteuses.

**Boîte noire:** Certains modèles de machine learning, comme les réseaux de neurones profonds, sont considérés comme des "boîtes noires" car il est difficile d'expliquer exactement comment ils arrivent à leurs conclusions.

**Sur-apprentissage:** Un modèle de machine learning peut être trop spécialisé sur les données d'entraînement et ne pas généraliser bien à de nouvelles données.

**Coût informatique:** L'entraînement de modèles de machine learning peut nécessiter une grande puissance de calcul et des infrastructures coûteuses.

**Éthique:** L'utilisation du machine learning soulève des questions éthiques importantes, notamment en ce qui concerne la protection de la vie privée, la discrimination et l'utilisation des données.

# MACHINE LEARNING
# (APPRENTISSAGE AUTOMATIQUE)

## Les applications du machine learning

Le machine learning est utilisé dans de nombreux domaines :

- **La reconnaissance d'images:** identification de visages, de textes dans des images, etc.
- **Le traitement du langage naturel:** traduction automatique, analyse de sentiment, chatbots.
- **Les recommandations:** produits, films, musique.
- **La prédiction:** prévision des ventes, d
- étection de fraudes, etc.
- **La conduite autonome:** les voitures autonomes utilisent le machine learning pour prendre des décisions en temps réel.

## Outils à utiliser:

- **Python:** Langage de programmation le plus utilisé en machine learning
- **Bibliothèques:** NumPy, Pandas, Scikit-learn, Seaborn…
- **Environnements de développement:** Jupyter Notebook

# MACHINE LEARNING

## Exemples Concrets d'Utilisation du Machine Learning

### Recommandations personnalisées

- **Netflix, Amazon, Spotify:** Ces plateformes utilisent le machine learning pour analyser vos habitudes de visionnage, d'achat ou d'écoute, afin de vous recommander du contenu personnalisé.
- **Moteurs de recherche:** Google et Bing utilisent le machine learning pour classer les résultats de recherche en fonction de vos préférences et de votre historique de recherche.

### Reconnaissance d'images et de voix

- **Identification faciale:** Les smartphones utilisent le machine learning pour déverrouiller votre téléphone en reconnaissant votre visage.
- **Reconnaissance vocale:** Les assistants virtuels comme Siri, Alexa et Google Assistant utilisent le machine learning pour comprendre et répondre à vos commandes vocales.
- **Détection d'objets:** Les voitures autonomes utilisent le machine learning pour détecter les piétons, les véhicules et les obstacles sur la route.

# MACHINE LEARNING

## Traitement du langage naturel

- **Chatbots:** Les chatbots utilisent le machine learning pour comprendre et répondre aux questions des utilisateurs.
- **Traduction automatique:** Les outils de traduction automatique, comme Google Translate, utilisent le machine learning pour traduire du texte d'une langue à une autre.
- **Analyse de sentiment:** Les entreprises utilisent le machine learning pour analyser les sentiments des clients à partir des commentaires et des avis en ligne.

## Finance

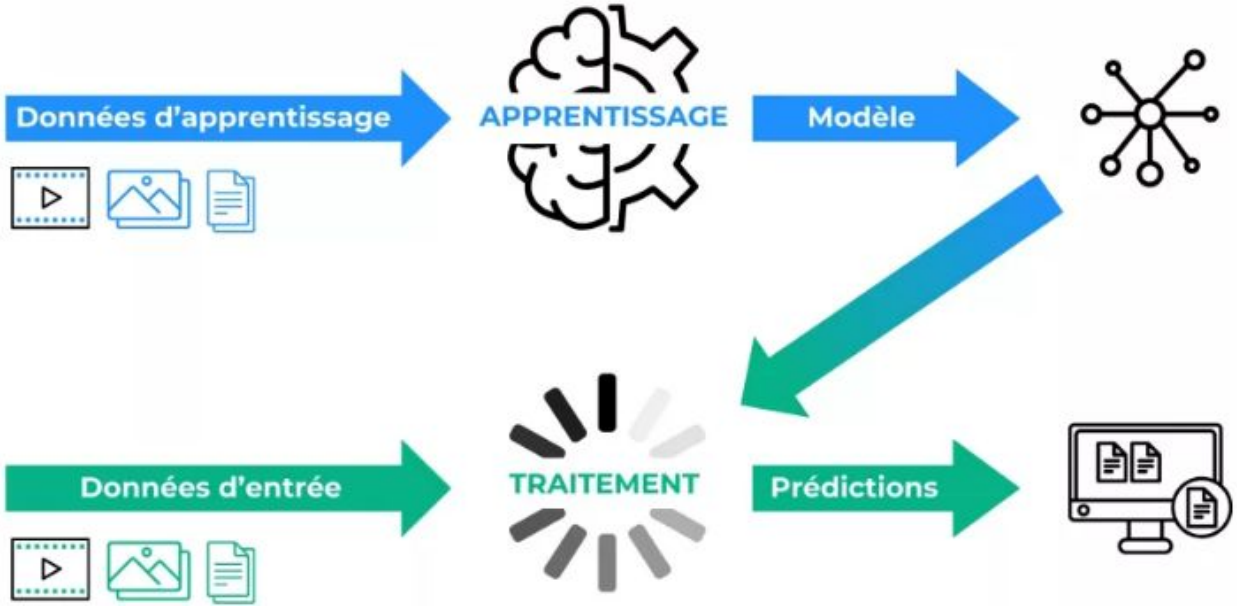
- **Détection de fraudes:** Les banques utilisent le machine learning pour détecter les transactions frauduleuses.
- **Prédiction de faillite:** Les entreprises utilisent le machine learning pour prédire la probabilité de faillite d'une entreprise.
- **Trading algorithmique:** Les traders utilisent le machine learning pour automatiser les décisions d'achat et de vente.

## Santé

- **Diagnostic médical:** Le machine learning peut être utilisé pour analyser des images médicales (radiographies, IRM) et détecter des anomalies.
- **Développement de médicaments:** Le machine learning peut aider à accélérer le processus de découverte de nouveaux médicaments.
- **Prédiction de maladies:** Les modèles de machine learning peuvent prédire le risque de développer certaines maladies.

# MACHINE LEARNING
# (APPRENTISSAGE AUTOMATIQUE)

## Machine Learning Processus:



# MACHINE LEARNING

## Machine Learning Processus:



monde réel

Collection /Stockage de données

Nettoyage /Organisation de données

Analyse de données

Machine Learning Model

# MACHINE LEARNING

## Machine Learning Processus:Préparation des données

1. Compréhension du problème (du point de vue réel): Quel problème souhaite-t-on résoudre avec le machine learning ?
2. Collecte des données pertinentes: Les données sont la matière première du machine learning. Il faut s'assurer qu'elles sont pertinentes, représentatives et de bonne qualité.
3. Nettoyage des données: Il s'agit de corriger les erreurs, les valeurs manquantes, les incohérences dans les données.
4. Transformation des données: Les données peuvent être transformées pour faciliter leur utilisation par les algorithmes (normalisation, standardisation, encodage).
5. Sélection des caractéristiques: On sélectionne les caractéristiques les plus pertinentes pour la prédiction.

# MACHINE LEARNING

## Machine Learning Processus: Modèle de Machine Learning

1. **Choix du Modèle**
   - **Choix de l'algorithme:** Le choix de l'algorithme dépend du type de problème (régression, classification, clustering, etc.) et de la nature des données.
   - **Configuration des hyperparamètres:** Les hyperparamètres sont des paramètres qui contrôlent le comportement de l'algorithme.
2. **Entraînement du modèle**
   - **Division des données:** Les données sont divisées en un ensemble d'entraînement et un ensemble de test.
   - **Entraînement:** L'algorithme apprend à partir des données d'entraînement.
   - **Évaluation:** Le modèle est évalué sur l'ensemble de test pour mesurer sa performance.
3. **Évaluation du modèle**
   - **Choix des métriques:** Les métriques d'évaluation dépendent du problème (accuracy, précision, rappel, F1-score, etc.).
   - **Analyse des résultats:** On analyse les résultats pour déterminer si le modèle est suffisamment performant.

# MACHINE LEARNING

## Machine Learning Processus: Déploiement et Visualisation

### Déploiement du modèle

- **Intégration dans une application:** Le modèle est intégré dans une application pour être utilisé en production.
- **Maintenance:** Le modèle doit être maintenu et mis à jour régulièrement pour s'adapter à l'évolution des données.

### Visualisation des résultats

- **Visualisation des données:** Des outils de visualisation permettent de mieux comprendre les données et les résultats du modèle.
- **Communication des résultats:** Les résultats sont présentés de manière claire et concise aux parties prenantes.

**En résumé**, un projet de machine learning est un processus itératif qui nécessite une bonne compréhension des données, des algorithmes et des outils. Chaque étape est cruciale pour obtenir un modèle performant et fiable.

# MACHINE LEARNING

## Machine Learning Processus: Résultats

**Prédictions numériques:** Les modèles de machine learning peuvent prédire des valeurs numériques, comme le prix d'une maison, la température maximale d'un jour ou la probabilité qu'un client achète un produit.

**Classification:** Les modèles de machine learning peuvent classer des données en catégories. Par exemple, ils peuvent déterminer si un email est un spam ou non, ou identifier le type d'objet présent dans une image.

**Clustering:** Les modèles de machine learning peuvent regrouper des données similaires en clusters. Cela peut être utilisé pour segmenter des clients, découvrir de nouveaux marchés ou identifier des anomalies.

**Génération de contenu:** Les modèles de machine learning peuvent générer de nouveaux contenus, comme des textes, des images ou de la musique.

**Détection d'anomalies:** Les modèles de machine learning peuvent identifier les événements ou les données qui s'écartent de la normale, ce qui est utile pour la détection de fraudes ou la maintenance prédictive.

# Définitions et concepts de base

# Définitions et concepts de base

### 1. Apprentissage

- **Supervisé:** Le modèle est entraîné sur des données étiquetées. Par exemple, pour reconnaître des chats, on lui montre des images de chats étiquetées "chat" et des images de chiens étiquetées "chien". Le modèle apprend à associer les caractéristiques des images aux étiquettes correspondantes.
- **Non supervisé:** Le modèle explore les données sans étiquettes pour trouver des structures ou des patterns cachés. Par exemple, il peut regrouper des clients en fonction de leurs comportements d'achat.
- **Par renforcement:** Le modèle apprend en interagissant avec un environnement. Il reçoit des récompenses ou des pénalités en fonction de ses actions et ajuste sa stratégie pour maximiser les récompenses.

# Définitions et concepts de base

## 1. Apprentissage

### Trois classes de problèmes d'apprentissage

#### Apprentissage Supervisé

**Données :** (*x*, *y*)
*x* est une donnée d'entrée, *y* est une étiquette (par exemple, une photo avec l'étiquette "chat")

**Objectif:** Apprendre à associer l'entrée à la sortie, c'est-à-dire **x -> y**

**Un exemple:** pour classifier Ceci est un chat



Un chat

#### Apprentissage Non Supervisé

**Données:** *x*,
*x* est une donnée, il n'y a pas d'étiquettes !

**Objectif:** Apprendre la structure sous-jacente des données.

**Un exemple:** Comparaison, regroupement



Ces deux choses se ressemblent

#### Apprentissage par Renforcement

**Données:** pas de données, seulement des paires état-action *(s, a)*

**Objectif:** Maximiser la récompense future sur plusieurs étapes temporelles.

**Un exemple:** récompense = joie



L'interaction avec le chat donne de la joie

# Définitions et concepts de base

**2. Modèle**

- **Représentation mathématique:** Un modèle de machine learning est une représentation mathématique des données. Il peut prendre différentes formes, comme une équation linéaire, un arbre de décision ou un réseau de neurones.
- **Paramètres:** Les paramètres d'un modèle sont les valeurs numériques qui sont ajustées pendant l'entraînement pour minimiser l'erreur.
- **Hyperparamètres:** Les hyperparamètres sont des paramètres qui contrôlent le processus d'apprentissage, comme le taux d'apprentissage ou le nombre d'itérations.

# Définitions et concepts de base

## 2. Modèle

- **Représentation mathématique:** Un modèle de machine learning est une représentation mathématique des données. Il peut prendre différentes formes, comme une équation linéaire, un arbre de décision ou un réseau de neurones.
- **Paramètres:** Les paramètres d'un modèle sont les valeurs numériques qui sont ajustées pendant l'entraînement pour minimiser l'erreur.
- **Hyperparamètres:** Les hyperparamètres sont des paramètres qui contrôlent le processus d'apprentissage, comme le taux d'apprentissage ou le nombre d'itérations.

# Définitions et concepts de base

## 3. Données

- **Données d'entraînement:** Les données utilisées pour entraîner le modèle.
- **Données de test:** Les données utilisées pour évaluer les performances du modèle.
- **Qualité des données:** La qualité des données est cruciale pour la performance du modèle. Les données doivent être propres, complètes et représentatives.

# Définitions et concepts de base

## 4. Entraînement

- **Fonction de coût:** Une fonction qui mesure l'erreur entre les prédictions du modèle et les valeurs réelles.
- **Optimisation:** Le processus d'ajustement des paramètres du modèle pour minimiser la fonction de coût.
- **Algorithmes d'optimisation:** Des algorithmes comme la descente de gradient sont utilisés pour optimiser les paramètres.

# Définitions et concepts de base

## 5. Évaluation

- **Métriques d'évaluation:** Des mesures comme la précision, le rappel, la F1-score, l'erreur quadratique moyenne sont utilisées pour évaluer les performances du modèle.
- **Matrice de confusion:** Une table qui montre les résultats d'un modèle de classification.

## 6. Overfitting et Underfitting *(Sur-apprentissage et Sous-apprentissage)*

- **Overfitting:** Un modèle qui est trop complexe et qui mémorise les données d'entraînement au lieu de généraliser.
- **Underfitting:** Un modèle qui est trop simple et qui ne capture pas les complexités des données.

# Définitions et concepts de base

## 7. Algorithmes

- **Régression linéaire:** Pour prédire une valeur numérique.
- **Arbres de décision:** Pour la classification et la régression.
- **Forêts aléatoires:** Un ensemble d'arbres de décision pour améliorer la précision.
- **Réseaux de neurones:** Inspirés du cerveau humain, ils sont particulièrement efficaces pour des tâches complexes comme la reconnaissance d'images et le traitement du langage naturel.
- **Support Vector Machines (SVM):** Pour la classification et la régression.

# Les Types d’Apprentissage

## Apprentissage Supervisé

## Supervised ML

# Apprentissage Supervisé

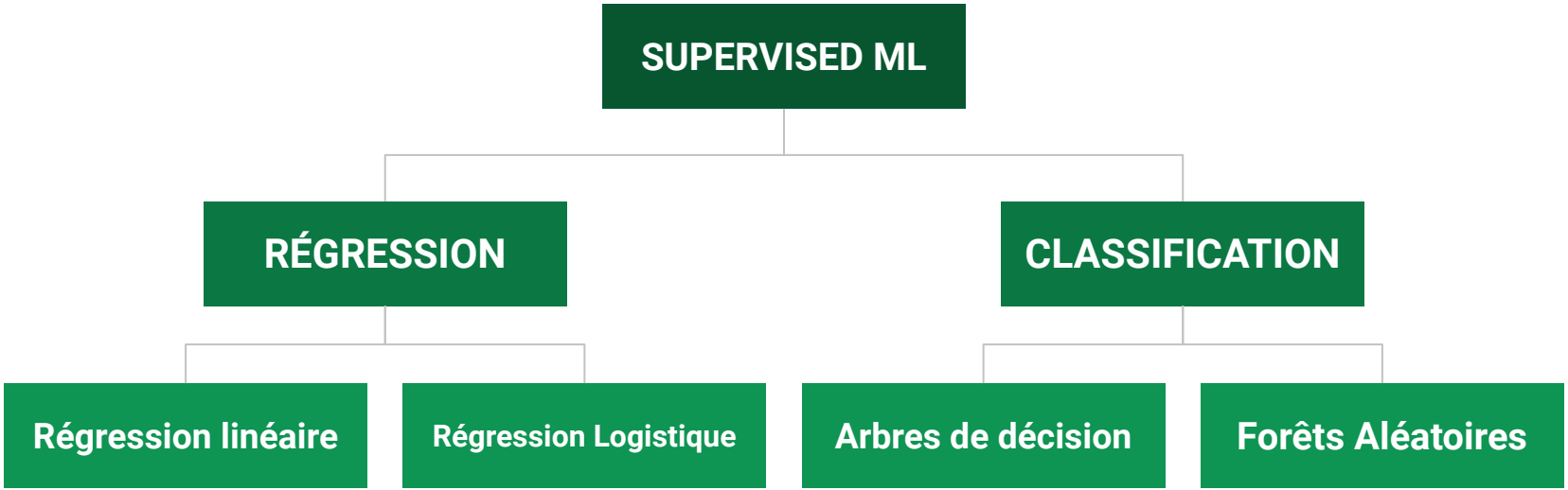
Un algorithme apprend à partir de données étiquetées. Imaginez que vous appreniez à reconnaître des animaux ou les différents noms de fruits en regardant des images avec leur nom écrit dessous. L'algorithme fonctionne de la même manière : on lui donne des exemples avec la bonne réponse, et il apprend à associer les caractéristiques des données (par exemple, la forme, la couleur) à la réponse correcte.

Suivant les étapes déjà mentionnées:

1. Collecte de données
2. Préparation des données
3. Choix du modèle
4. Entraînement du modèle
5. Évaluation du modèle

# Apprentissage Supervisé

## Algorithmes de Supervised ML



# RÉGRESSION

- La régression en apprentissage supervisé est une technique statistique qui vise à modéliser la relation entre une **variable dépendante** (**la variable à prédire**) et une ou plusieurs **variables indépendantes** (**les prédicteurs**).
- Le but est de construire un modèle mathématique qui permet de prédire la valeur de la variable dépendante à partir des valeurs des variables indépendantes.
- Le choix du modèle de régression dépend de la nature des données, de la relation entre les variables et de l'objectif de l'analyse.

# RÉGRESSION

**Voici les points clés à retenir :**

- **Variable dépendante:** La variable que l'on cherche à prédire.
- **Variables indépendantes:** Les variables qui influencent la variable dépendante.
- **Modèle mathématique:** Une équation qui décrit la relation entre les variables.
- **Estimation des paramètres:** Le processus de détermination des meilleurs coefficients pour le modèle.
- **Prédiction:** L'utilisation du modèle pour estimer de nouvelles valeurs de la variable dépendante.

# RÉGRESSION LINÉAIRE

***Un pré-requis en Algèbre Linéaire est fortement souhaité pour ce chapitre***

**Définition:**

Un Fondement de l'Apprentissage Supervisé,Utilisée pour établir une relation linéaire entre une variable dépendante (que l'on cherche à prédire) et une ou plusieurs variables indépendantes (les prédicteurs).

En d'autres termes, elle cherche à trouver la droite (ou l'hyperplan dans un espace à plus de deux dimensions) qui ajuste au mieux les données.

# RÉGRESSION LINÉAIRE

1. **Hypothèse linéaire:** On suppose qu'il existe une relation linéaire entre la variable à prédire (notée généralement Y) et les variables explicatives (notées X1, X2, ... Xn). Cette relation peut s'écrire sous la forme d'une équation :

```
Y = β0 + β1*X1 + β2*X2 + ... + βn*Xn + ε
```

- Y : variable à prédire
- X1, X2, ..., Xn : variables explicatives
- β0, β1, ..., βn : coefficients à estimer
- ε : terme d'erreur

# RÉGRESSION LINÉAIRE

2. **Estimation des coefficients:** L'objectif est de trouver les valeurs des coefficients β qui minimisent la somme des carrés des erreurs entre les valeurs prédites par le modèle et les valeurs réelles. Cette méthode est appelée méthode des moindres carrés ordinaires.

3. **Prédiction:** Une fois les coefficients estimés, on peut utiliser l'équation de régression pour prédire la valeur de Y pour de nouvelles données.

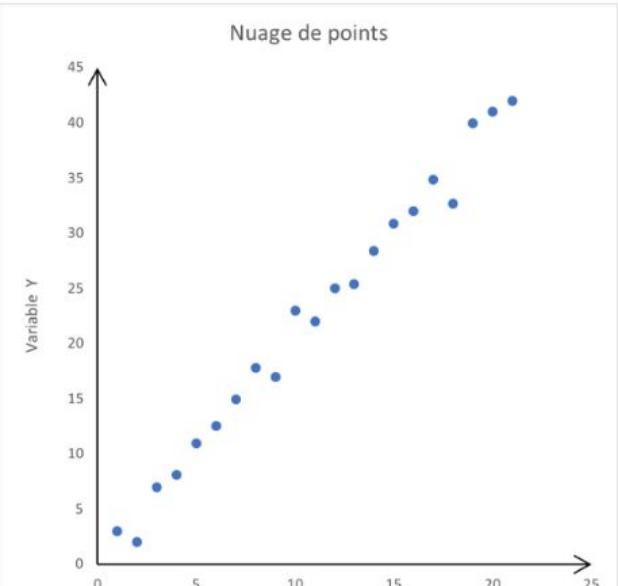
**Représentation graphique**

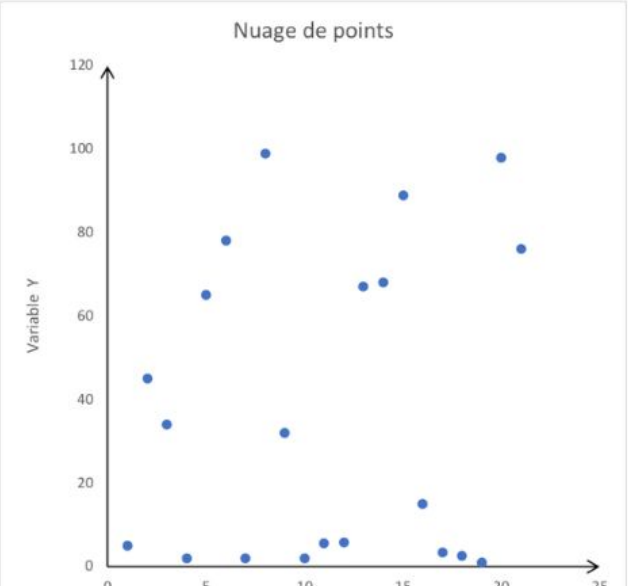
[Image d'une régression linéaire simple avec une droite ajustée sur des données]

# RÉGRESSION LINÉAIRE

Avant de débuter une étude de régression simple, il faut d'abord tracer les observations .$(X_i , Y_i)$, $i=1,\ldots,p$.

Le but est de savoir si le modèle linéaire est oui ou non pertinent pour l'étude de notre phénomène. Le graphique est au départ un nuage de points et on relève la tendance qu'a la forme de ce nuage de points.



# RÉGRESSION LINÉAIRE

Au vu de ces deux graphiques, il semble approprié d'utiliser le modèle linéaire pour la première image et pas pour la deuxième qui ne laisse transparaître aucune tendance connue.

## Modélisation

| Modélisation | Nature de la régression |
|---|---|
| Une seule variable explicative $X$ | Régression simple |
| Plusieurs variables explicatives $X_j$ $(j=1,\ldots,q)$ | Régression multiples |

***Modèle simple:***

$Y=aX+b+\varepsilon$ où $f(X)= aX+b$

- $Y$, la variable cible, aléatoire dépendante
- $a$ et $b$, les coefficients (pente et ordonnée à l'origine) à estimer
- $X$, la variable explicative, indépendante
- $\varepsilon$, une variable aléatoire qui représente l'erreur

# RÉGRESSION LINÉAIRE

## *Modèle multiple:*

### $Y=ax_1+bx_2+cx_3+\cdots+K+\varepsilon$ où $f(X)= aX+b$

- $Y$, la variable cible, aléatoire dépendante
- $a,\ldots,K$ les coefficients (pente et ordonnée à l'origine) à estimer
- $X=(x_1,\ldots,x_q)$, la variable explicative, indépendante
- $\varepsilon$, une variable aléatoire qui représente l'erreur

## *Modèle matriciel (sous forme de matrice):*

$$Y=AX+\varepsilon$$

Où

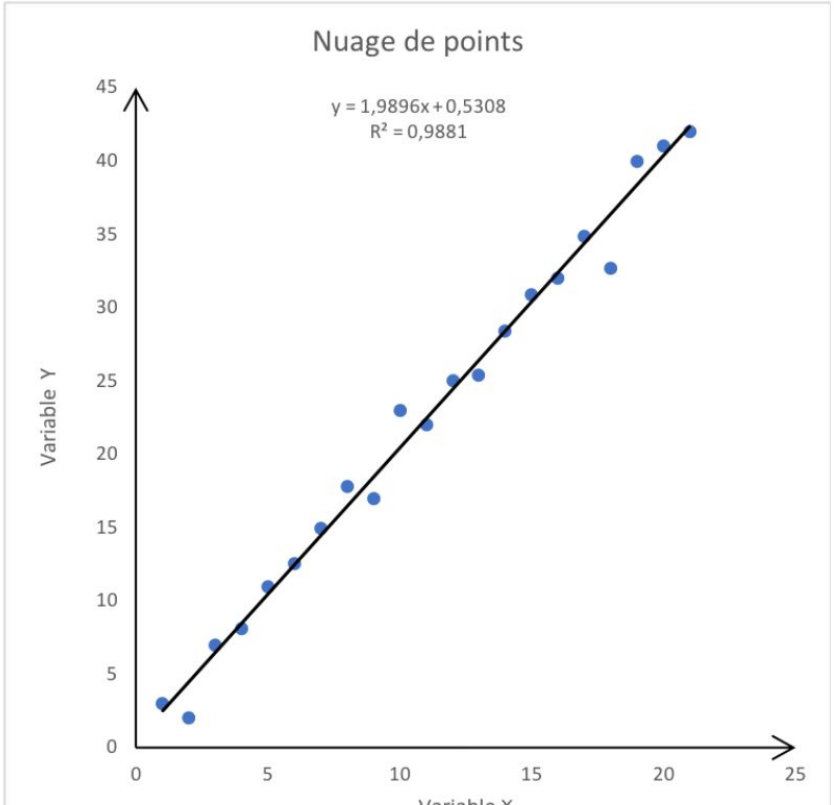
$$Y = \begin{pmatrix} y_1 \\ \vdots \\ y_p \end{pmatrix}, X = \begin{pmatrix} 1 & x_1 \\ & \vdots \\ 1 & x_p \end{pmatrix}, A = \begin{pmatrix} a_0 \\ a_1 \end{pmatrix} \text{ et } \varepsilon = \begin{pmatrix} \varepsilon_1 \\ \vdots \\ \varepsilon_p \end{pmatrix}$$

Avec :

- ***Y***, un vecteur à expliquer de taille ***n* x 1**,
- ***X***, la matrice explicative de taille ***n* x 2**,
- ***ε***, le vecteur d'erreurs de taille ***n* x 1**

# RÉGRESSION LINÉAIRE

## Visualisation: nuage de droite et droite de régression




C’est en confrontant l’ équation calculée par l’algorithme de régression linéaire aux nouvelles données de la réalité (X) que les prédictions (Y) seront réalisées par l’algorithme d’intelligence artificielle en production.

# RÉGRESSION LINÉAIRE

## Visualisation: nuage de droite et droite de régression

Le terme $R^2$ de l'image représente le coefficient de corrélation de Bravais-Pearson au carré. Ce coefficient mesure l'intensité de la relation linéaire entre $Y$ et $X$ .

Voici sa formule :

$$R = \frac{\sum_{i=1}^{p}(X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum_{i=1}^{p}(X_i - \bar{X})^2}\sqrt{\sum_{i=1}^{p}(Y_i - \bar{Y})^2}}$$

Le coefficient de corrélation est un nombre toujours compris entre -1 et 1.

- Si $R$ est proche de 1 : il y a une forte liaison linéaire entre les variables et les valeurs prises par $Y$ ont tendance à croître quand les valeurs de $X$ augmentent.
- Si $R$ est proche de 0 : il n'y a pas de liaison linéaire

# RÉGRESSION LINÉAIRE

## Visualisation: Résidus

- Les résidus sont les différences entre les valeurs observées et les valeurs prédites par le modèle. Un graphique des résidus par rapport aux valeurs prédites peut révéler des patterns ou des problèmes dans le modèle.
- **Homoscédasticité:** Les résidus doivent être distribués de manière constante autour de zéro.
- **Indépendance:** Les résidus ne doivent pas présenter de corrélation.

## Visualisation: Matrice de corrélation

- Pour des modèles à plusieurs variables explicatives, une matrice de corrélation permet de visualiser les relations linéaires entre toutes les variables.
- Une corrélation élevée entre deux variables explicatives peut indiquer un problème de multicollinéarité ( se produit lorsqu'il existe une corrélation linéaire forte entre deux ou plusieurs variables indépendantes dans un modèle de régression linéaire multiple. )

# RÉGRESSION LINÉAIRE

## Exemple concret de régression linéaire

### Prédire le prix d'une maison en fonction de sa superficie

**Données:** Nous disposons d'un dataset contenant les superficies (en mètres carrés) et les prix de vente (en euros) de plusieurs maisons.

**Modèle de régression linéaire:** Nous allons utiliser un modèle de régression linéaire simple pour prédire le prix d'une maison en fonction de sa superficie. Le modèle peut être représenté par l'équation suivante :

$$Prix = \beta_0 + \beta_1 * Superficie + \varepsilon$$

- `Prix`: La variable dépendante (ce que nous voulons prédire)
- `Superficie`: La variable indépendante (le prédicteur)
- $\beta_0$: L'intercept (valeur de `Prix` lorsque `Superficie` est 0)
- $\beta_1$: Le coefficient de régression (impact d'une augmentation de 1 m² sur le prix)
- $\varepsilon$: Le terme d'erreur, qui représente la part de la variation de `Prix` non expliquée par `Superficie`

```python
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split


# Charger les données
data = pd.read_csv("maisons.csv")

# Séparer les données en variables indépendantes (X) et dépendante (y)
X = data[['Superficie']]
y = data['Prix']

# Diviser les données en ensembles d'entraînement et de test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,


# Créer le modèle de régression linéaire
model = LinearRegression()

# Entraîner le modèle sur
 les données d'entraînement
model.fit(X_train, y_train)

# Faire des prédictions sur les données de test
y_pred = model.predict(X_test)


# Visualiser les résultats
plt.scatter(X_test, y_test, color='black')
plt.plot(X_test, y_pred, color='blue', linewidth=3)
plt.xlabel("Superficie (m²)")
plt.ylabel("Prix (euros)")
plt.title("Régression linéaire : Prix en fonction de la superficie")
plt.show()
```

# RÉGRESSION LINÉAIRE

## Exemple concret de régression linéaire

### Prédire le prix d'une maison en fonction de sa superficie

`plt.scatter(X_test, y_test, color='black', label="Données réelles")`**:** Cette ligne ajoute les points de données réelles (les points noirs) au graphique.

`plt.plot(X_test, y_pred, color='blue', linewidth=3, label="Droite de régression")`**:** Cette ligne trace la droite de régression (la ligne bleue) qui représente les prédictions du modèle.

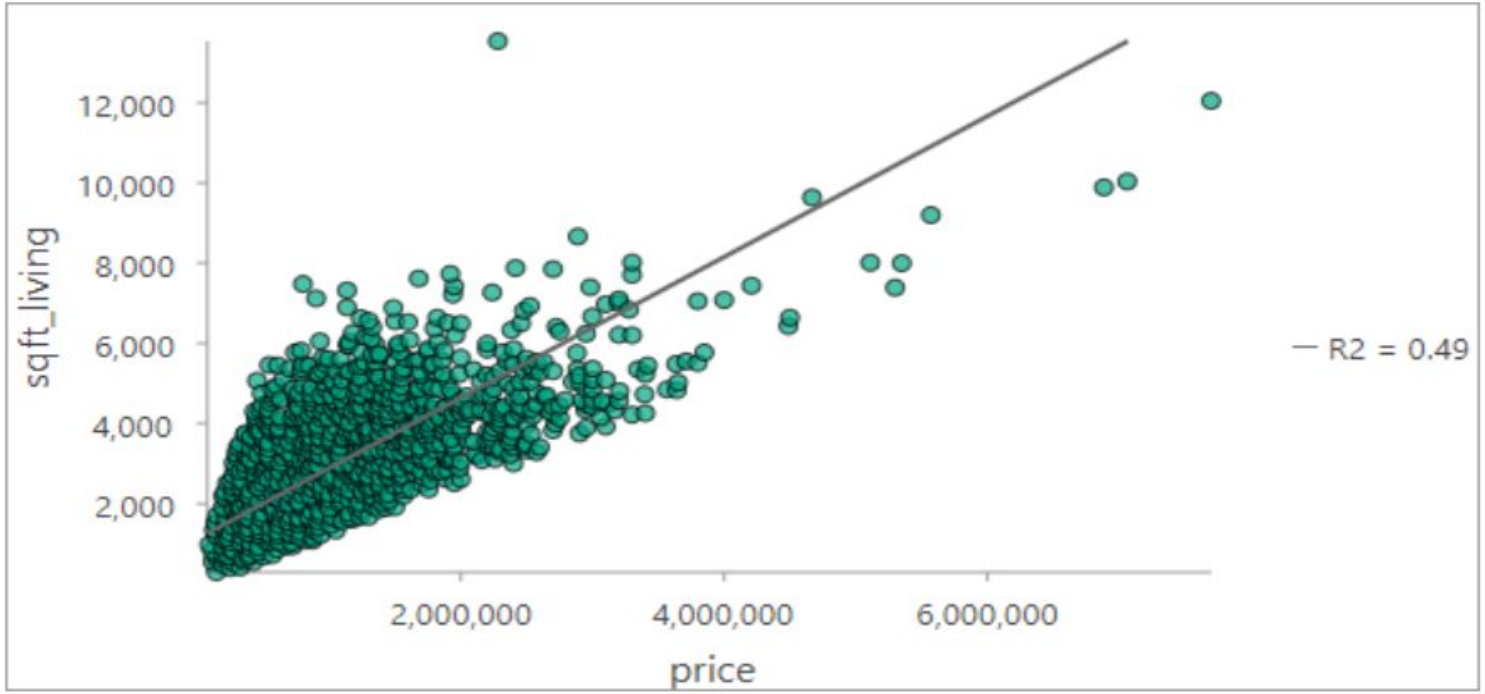
`plt.legend()`**:** Cette ligne ajoute une légende pour différencier les points de données réelles et la droite de régression.

Vous verrez un nuage de points représentant les données réelles (prix en fonction de la superficie) et une ligne droite qui représente la prédiction de votre modèle de régression linéaire.

# RÉGRESSION LINÉAIRE

## Exemple concret de régression linéaire

### Prédire le prix d'une maison en fonction de sa superficie



# RÉGRESSION LOGISTIQUE

## Définition:

La régression logistique est une méthode statistique utilisée pour modéliser la relation entre une variable dépendante binaire (ayant deux valeurs possibles, par exemple 0 ou 1, oui ou non) et un ensemble de variables indépendantes. Elle est particulièrement utile pour les problèmes de classification, où l'objectif est de prédire à quelle classe appartient une observation.

Un modèle de régression logistique permet aussi de prédire la probabilité qu'un événement arrive (valeur de 1) ou non (valeur de 0) à partir de l'optimisation des coefficients de régression. Ce résultat varie toujours entre 0 et 1. Lorsque la valeur prédite est supérieure à un seuil, l'événement est susceptible de se produire, alors que lorsque cette valeur est inférieure au même seuil, il ne l'est pas.

# RÉGRESSION LOGISTIQUE

## Fonctionnement:

**Fonction logistique:** Au lieu d'une équation linéaire directe, la régression logistique utilise une fonction logistique (ou sigmoïde) qui contraint les valeurs de prédiction entre 0 et 1, ce qui correspond à des probabilités.

**Estimation des coefficients:** Les coefficients de la régression logistique sont estimés en maximisant la vraisemblance, ce qui signifie qu'on cherche les paramètres qui rendent les données observées les plus probables.

**Interprétation des coefficients:** Les coefficients de régression logistique indiquent l'effet d'une unité de changement dans une variable indépendante sur le logarithme des odds (rapport des chances) de l'événement.

# RÉGRESSION LOGISTIQUE

## Fondement Mathématique:

Le cœur de la régression logistique est la **fonction logistique** (ou fonction sigmoïde). Cette fonction prend en entrée n'importe quelle valeur réelle et renvoie une valeur comprise entre 0 et 1, ce qui est idéal pour représenter une probabilité.

La fonction logistique est définie comme suit :

$$\sigma(z) = 1 / (1 + e^{(-z)})$$

Où :

- $z$ est une combinaison linéaire des variables explicatives : $z = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_p X_p$
- $\beta_0$, $\beta_1$, ... $\beta_p$ sont les coefficients de régression à estimer
- $\sigma(z)$ est la probabilité estimée

# RÉGRESSION LOGISTIQUE

## Le Modèle de Régression Logistique:

Le modèle de régression logistique relie la probabilité d'un événement à un ensemble de variables prédictives. On peut l'exprimer ainsi :

$$P(Y=1 \mid X) = \sigma(\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_p X_p)$$

Où :

- $P(Y=1 \mid X)$ est la probabilité que la variable dépendante $Y$ soit égale à 1 (succès) étant donné les valeurs des variables explicatives $X$.

### Estimation des Coefficients

Pour estimer les coefficients $\beta$, on utilise généralement la **méthode du maximum de vraisemblance**. Cette méthode consiste à trouver les valeurs des coefficients qui maximisent la probabilité d'observer les données réelles, étant donné le modèle.

# RÉGRESSION LOGISTIQUE

## Utilisation de Régression Logistique:

**Quand utiliser la régression logistique ?**

- **Classification binaire:** Prédire si un individu aura une maladie ou non, si un client va acheter un produit ou non, etc.
- **Modélisation de la probabilité:** Estimer la probabilité qu'un événement se produise.
- **Variables indépendantes continues ou catégorielles:** La régression logistique peut gérer à la fois des variables indépendantes continues et catégorielles.

**Exemple : Prédiction de l'admission à l'université**

Imaginez que vous souhaitez prédire si un étudiant sera admis à l'université en fonction de ses notes au baccalauréat et de son score au test d'admission. La variable dépendante (admission) est binaire (0 : non admis, 1 : admis). La régression logistique vous permettra de modéliser la probabilité d'admission en fonction des notes et du score.

# RÉGRESSION LOGISTIQUE

## Utilisation de Régression Logistique:

le Dataset contiendra surement les colonnes suivantes:

**Admission:** Variable cible (0 ou 1).

**Notes au baccalauréat:** Note obtenue au baccalauréat.

**Score au test d'admission:** Score obtenu à un test standardisé (ex : SAT, GRE).

**Rang de classe:** Position de l'étudiant dans sa classe.

**Nombre d'activités extra-scolaires:** Nombre d'activités auxquelles l'étudiant a participé.

**Recommandation des enseignants:** Note attribuée par les enseignants (sur une échelle)

**Équation:** La régression logistique utilise une équation de la forme :

$$P(\text{Admission} = 1) = 1 / (1 + \exp(-(\beta_0 + \beta_1 * \text{Notes} + \beta_2 * \text{Score} + \ldots)))$$

# RÉGRESSION LOGISTIQUE

ù :

- `P(Admission = 1)` est la probabilité d'admission.
- $\beta_0$ , $\beta_1$ , $\beta_2$ , ... sont les coefficients du modèle à estimer.
- `Notes` , `Score` , ... sont les variables prédictives.

**nterprétation des coefficients:**

- Un coefficient positif indique qu'une augmentation de la variable correspondante augmente la probabilité d'admission.
- Un coefficient négatif indique qu'une augmentation de la variable correspondante diminue la probabilité d'admission.

**euil de décision:** On fixe généralement un seuil de 0.5. Si la probabilité prédite est upérieure à 0.5, on considère que l'étudiant sera admis.

# RÉGRESSION LOGISTIQUE

Processus:

1. **Collecte des données:** Les données peuvent être obtenues auprès de l'université ou d'autres sources.
2. **Prétraitement des données:** Nettoyer les données, gérer les valeurs manquantes, encoder les variables catégorielles (si nécessaire).
3. **Séparation des données:** Diviser les données en un ensemble d'entraînement et un ensemble de test.
4. **Entraînement du modèle:** Utiliser l'ensemble d'entraînement pour estimer les coefficients du modèle.
5. **Évaluation du modèle:** Évaluer la performance du modèle sur l'ensemble de test à l'aide de métriques comme l'accuracy, la précision, le rappel, la F1-score, la matrice de confusion, etc.
6. **Interprétation des résultats:** Analyser les coefficients du modèle pour comprendre l'importance de chaque variable prédictive et faire des prédictions.

# RÉGRESSION LOGISTIQUE

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score


# Charger les données
data = pd.read_csv("data_admission.csv")

# Séparer les variables
X = data[['notes', 'score_test']]
y = data['admission']

# Division en ensembles d'entraînement et de test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,

# Créer
 et entraîner le modèle
model = LogisticRegression()
model.fit(X_train, y_train)

# Faire des prédictions
y_pred = model.predict(X_test)

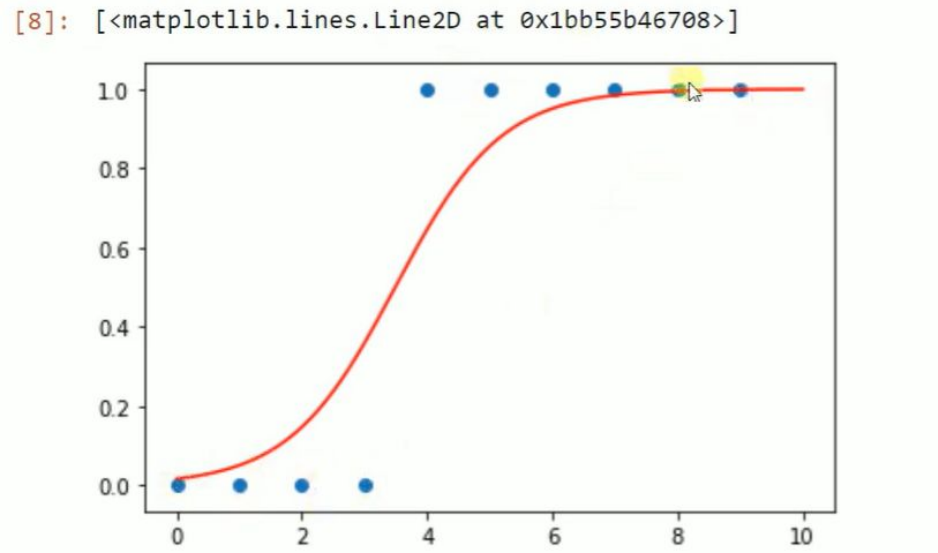
# Évaluer le modèle
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

# RÉGRESSION LOGISTIQUE

## Utilisation de Régression Logistique:

### Interprétation des résultats :

- **Accuracy:** Indique le pourcentage de prédictions correctes.
- **Coefficients du modèle:** Les coefficients associés à chaque variable prédictive indiquent l'impact de cette variable sur la probabilité d'admission. Un coefficient positif signifie qu'une augmentation de cette variable augmente la probabilité d'admission.



# Évaluation des Modèles de Machine Learning Supervisé

Elle permet de mesurer la performance du modèle sur un ensemble de données de test, ce qui nous aide à comprendre sa capacité de généralisation à de nouvelles données.

## Métriques d'Évaluation

### 1. Métriques pour la Régression

**Mean Squared Error (MSE):** Moyenne des carrés des erreurs entre les valeurs prédites et les valeurs réelles.

**Root Mean Squared Error (RMSE):** Racine carrée de la MSE.

**Mean Absolute Error (MAE):** Moyenne de la valeur absolue des erreurs.

**Mean Absolute Percentage Error (MAPE):** Moyenne du pourcentage d'erreur pour chaque prédiction.

# Évaluation des Modèles de Machine Learning Supervisé

## 2. Métriques pour la classification

- **Matrice de confusion:**

**- True Positive (TP):** Nombre de cas positifs correctement prédits.

**- True Negative (TN):** Nombre de cas négatifs correctement prédits.

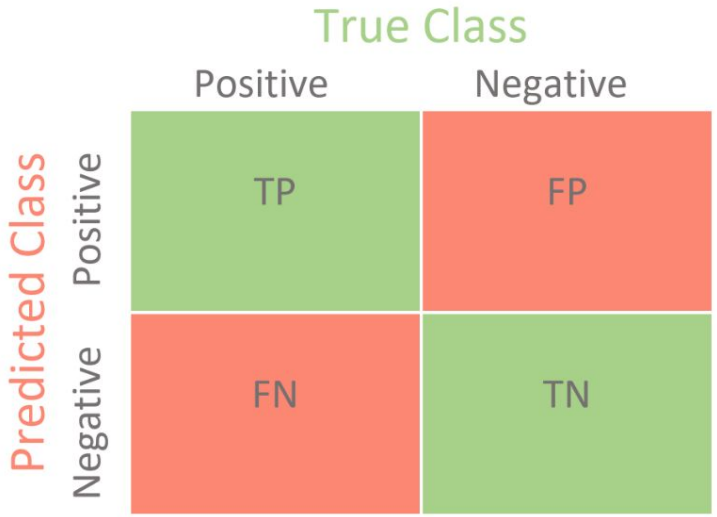
**- False Positive (FP):** Nombre de cas négatifs incorrectement prédits comme positifs (faux positif).

**- False Negative (FN):** Nombre de cas positifs incorrectement prédits comme négatifs (faux négatif).

# Évaluation des Modèles de Machine Learning Supervisé

- **Matrice de confusion:**

```
from sklearn import metrics
confusion_matrix = metrics.confusion_matrix(actual, predicted)
```



# Évaluation des Modèles de Machine Learning Supervisé

- **Accuracy**
  - Proportion de prédictions correctes.
  - `Accuracy = (TP + TN) / (TP + TN + FP + FN)`
- **Précision**
  - Proportion de prédictions positives correctes parmi toutes les prédictions positives.
  - `Precision = TP / (TP + FP)`
- **Recall**
  - Proportion de vrais positifs identifiés par le modèle.
  - `Recall = TP / (TP + FN)`

# Évaluation des Modèles de Machine Learning Supervisé

- **F1-Score**

- Moyenne harmonique de la précision et du rappel.
- `F1-score = 2 * (Precision * Recall) / (Precision + Recall)`

- **AUC-ROC (Area Under the Curve - Receiver Operating Characteristic):**

Mesure la capacité du modèle à distinguer entre les classes positives et négatives.

# ARBRES DE DÉCISION

Un arbre de décision est un modèle de machine learning qui imite le processus de décision humaine en se basant sur une série de règles. Il est particulièrement utile pour les problèmes de classification et de régression.

## Structure d'un arbre de décision :

- **Nœuds:** Représentent les attributs (ou features) sur lesquels la décision est prise.
- **Branches:** Représentent les issues possibles d'un test sur un nœud.
- **Feuilles:** Représentent les classes (pour la classification) ou les valeurs (pour la régression).

## Principe de fonctionnement:

L'algorithme parcourt l'arbre à partir de la racine jusqu'à une feuille, en prenant des décisions à chaque nœud en fonction de la valeur de l'attribut. La feuille atteinte correspond à la prédiction finale.

# ARBRES DE DÉCISION

## Pourquoi utiliser les arbres de décision ?

- **Interprétation facile:** Les règles de décision sont facilement compréhensibles par les humains.
- **Traitement des données catégorielles et numériques:** Les arbres de décision peuvent gérer à la fois des données numériques et catégorielles.
- **Peu sensible aux outliers:** Les arbres de décision sont relativement robustes aux valeurs aberrantes.

## Avantages et inconvénients

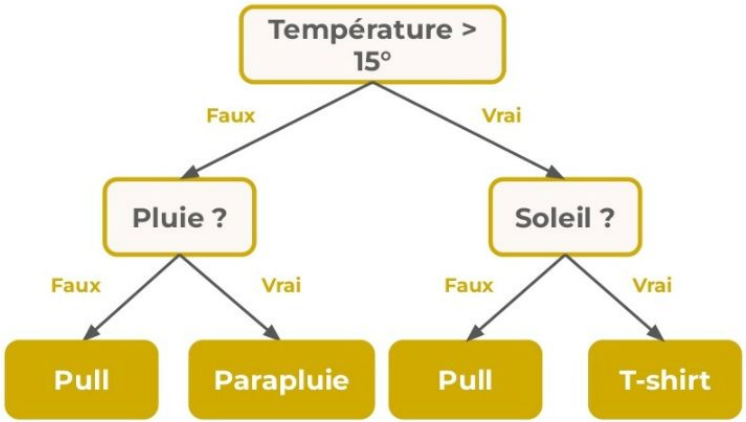
- **Avantages:**
  - Interprétation facile
  - Traitement de données hétérogènes
  - Peu sensible aux outliers
  - Rapide à entraîner
- **Inconvénients:**
  - Sensible au sur-apprentissage
  - Peu performant sur des données très bruitées
  - Peut créer des arbres complexes difficiles à visualiser

# ARBRES DE DÉCISION

## Exemples Concrets d'Arbres de Décision

### 1. Prédiction de la météo

- **Variables d'entrée:** Température, humidité, pression atmosphérique, vitesse du vent, etc.
- **Variable cible:** Pluie (oui/non)
- **Règles de décision:**
  - Si la température est inférieure à 10 degrés Celsius ET l'humidité est supérieure à 80%, alors il pleut.
  - Si la pression atmosphérique est en baisse ET la vitesse du vent est supérieure à 20 km/h, alors il y a une forte probabilité de pluie.

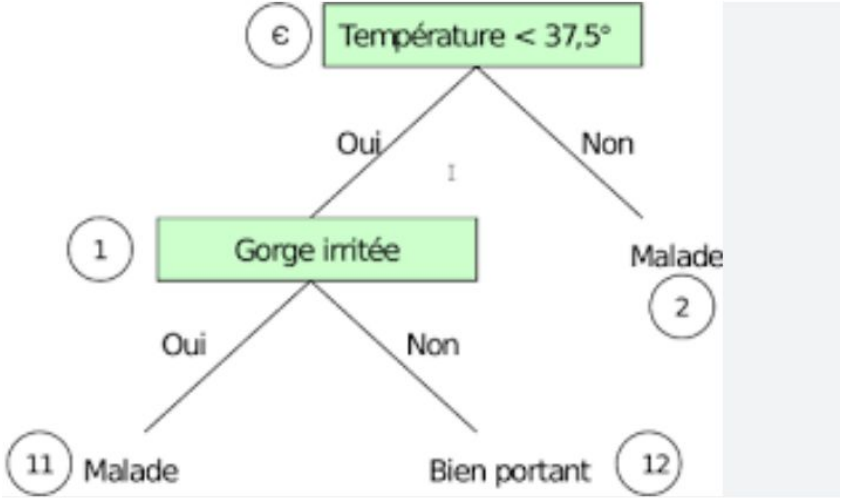


# ARBRES DE DÉCISION

## Exemples Concrets d'Arbres de Décision

**2. Diagnostic médical**

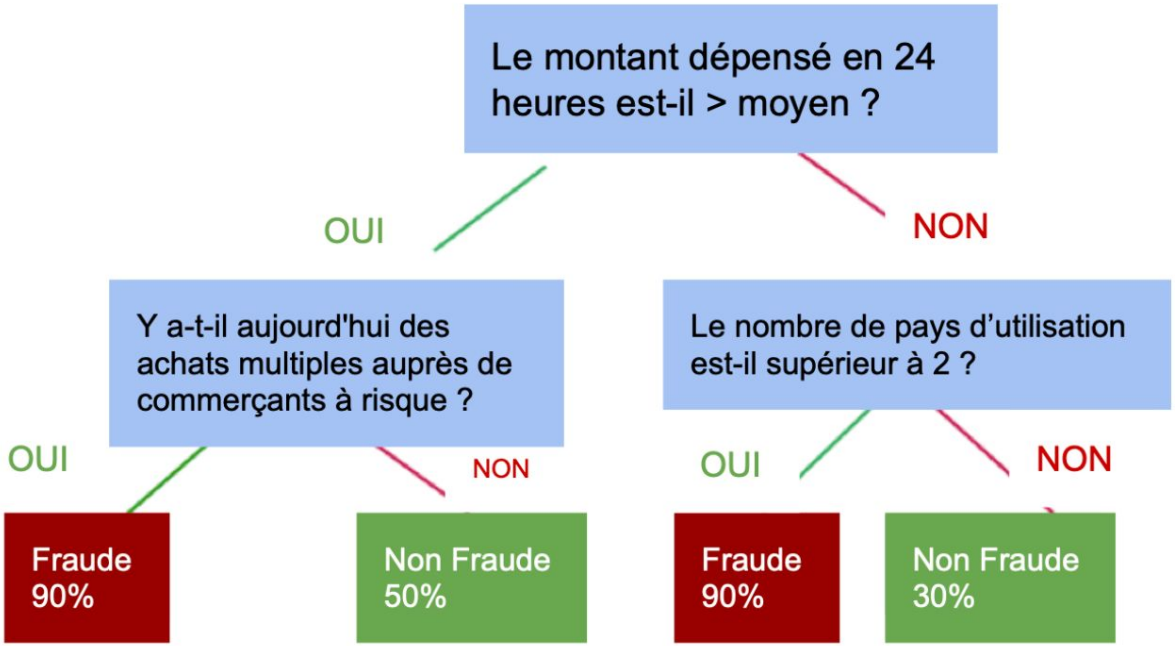
- **Variables d'entrée:** Symptômes, résultats de tests médicaux, antécédents médicaux.
- **Variable cible:** Maladie (oui/non) ou type de maladie.
- **Règles de décision:**
  - Si la température est élevée ET la toux est sèche, alors c'est probablement une infection virale.
  - Si la fièvre est élevée ET il y a des douleurs articulaires, alors c'est peut-être une grippe.



# ARBRES DE DÉCISION

## Exemples Concrets d'Arbres de Décision

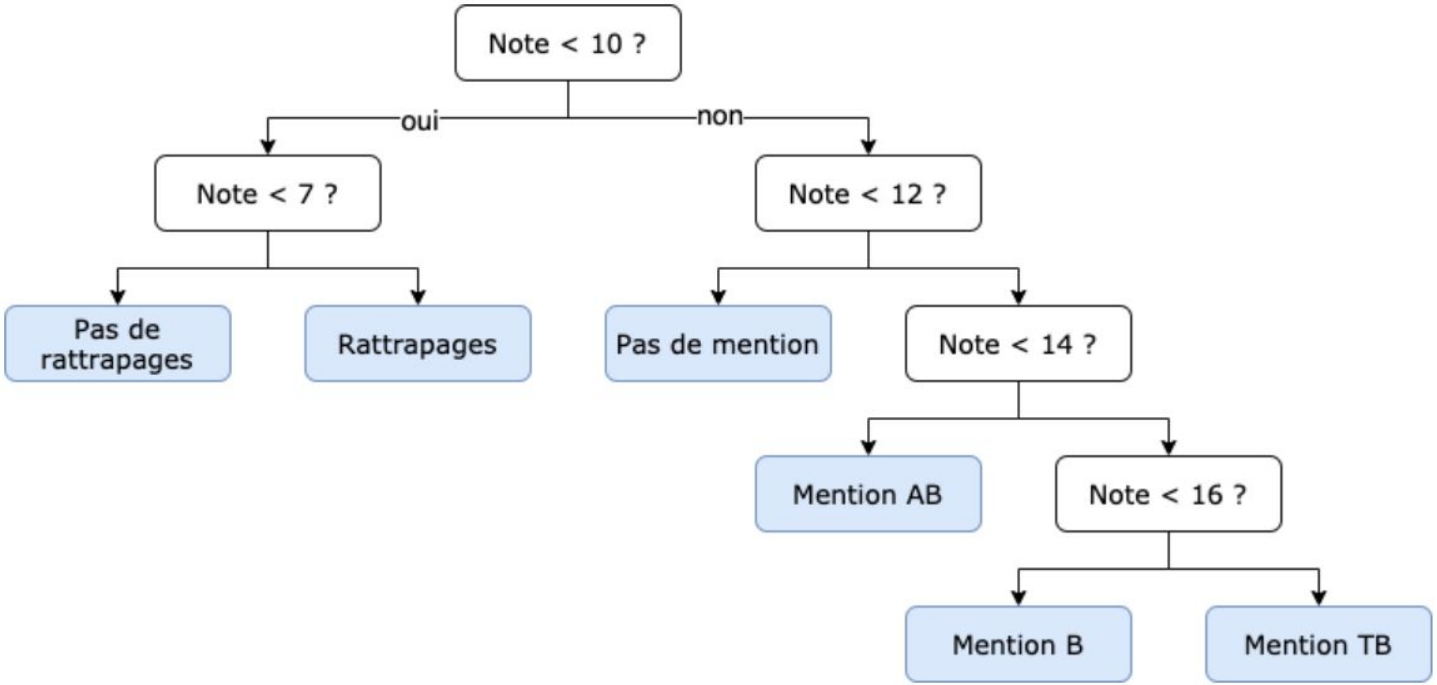
### 3. Finance : Détection de fraude



# ARBRES DE DÉCISION

## Exemples Concrets d'Arbres de Décision

### 3. Finance : Evaluation des résultats scolaires



# ARBRES DE DÉCISION

## Implémentation en Python avec scikit-learn

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Charger
 les données
X = ...  # Variables explicatives
y = ...  # Variable cible

# Diviser les données en ensembles d'entraînement et de test
X_train, X_test, y_train, y_test = train_test_split(X, y,
 test_size=0.2, random_state=42)


# Créer et entraîner l'arbre de décision
model = DecisionTreeClassifier()
model.fit(X_train, y_train)

# Faire des prédictions
y_pred = model.predict(X_test)

# Évaluer le modèle
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

# ARBRES DE DÉCISION

## Visualisation d'un arbre de décision

```
from sklearn import tree
import matplotlib.pyplot as plt

plt.figure(figsize=(20,10))
tree.plot_tree(model, filled=True,
feature_names=X.columns, class_names=['0', '1'])
plt.show()
```

# FORÊT ALÉATOIRE- RANDOM FOREST

## Définition:

C’est un algorithme d'apprentissage supervisé qui combine plusieurs arbres de décision.

Chaque arbre est construit sur un sous-échantillon aléatoire des données d'entraînement et en considérant un sous-ensemble aléatoire des caractéristiques.

La prédiction finale est obtenue par un vote majoritaire (pour la classification) ou une moyenne (pour la régression) de tous les arbres de la forêt.

**Pourquoi utiliser les Forêts Aléatoires ?**

- **Réduction du sur-apprentissage:** En créant de nombreux arbres sur des sous-échantillons différents, les forêts aléatoires réduisent le risque de sur-apprentissage, un problème courant avec les arbres de décision individuels.
- **Haute précision:** Les forêts aléatoires sont souvent très performantes en termes de précision, notamment sur des problèmes complexes avec de nombreuses variables.
- **Traitement de données hétérogènes:** Elles peuvent gérer à la fois des variables numériques et catégorielles.
- **Importance des variables:** Les forêts aléatoires permettent d'évaluer l'importance de chaque variable dans la prédiction.

# FORÊT ALÉATOIRE- RANDOM FOREST

## Quand utiliser les forêts aléatoires ?

- **Classification:** Prédiction de classes (spam/non-spam, fraude/non-fraude, etc.)
- **Régression:** Prédiction de valeurs numériques (prix d'une maison, durée de vie d'un moteur, etc.)
- **Grands ensembles de données:** Les forêts aléatoires peuvent gérer efficacement de grands volumes de données.
- **Données avec de nombreuses variables:** Elles sont robustes à la présence de variables non pertinentes.

# FORÊT ALÉATOIRE- RANDOM FOREST

## Exemples Concrets:

### 3. Prédiction de la Durée de Vie d'un Équipement Industriel

- **Problématique:** Estimer le temps avant la panne d'une machine.
- **Données:** Données de maintenance, mesures de performance, conditions d'utilisation, etc.
- **Utilisation:** Les forêts aléatoires peuvent aider à planifier la maintenance prédictive et à réduire les coûts associés aux pannes imprévues.

### 4. Analyse de Sentiment

- **Problématique:** Déterminer si un texte exprime une opinion positive, négative ou neutre.
- **Données:** Mots, phrases d'un texte.
- **Utilisation:** Les forêts aléatoires peuvent être utilisées pour analyser des commentaires clients, des avis en ligne, ou des publications sur les réseaux sociaux.

# FORÊT ALÉATOIRE- RANDOM FOREST

## Exemples Concrets:

### 1. Prédiction de la fidélité d’un client

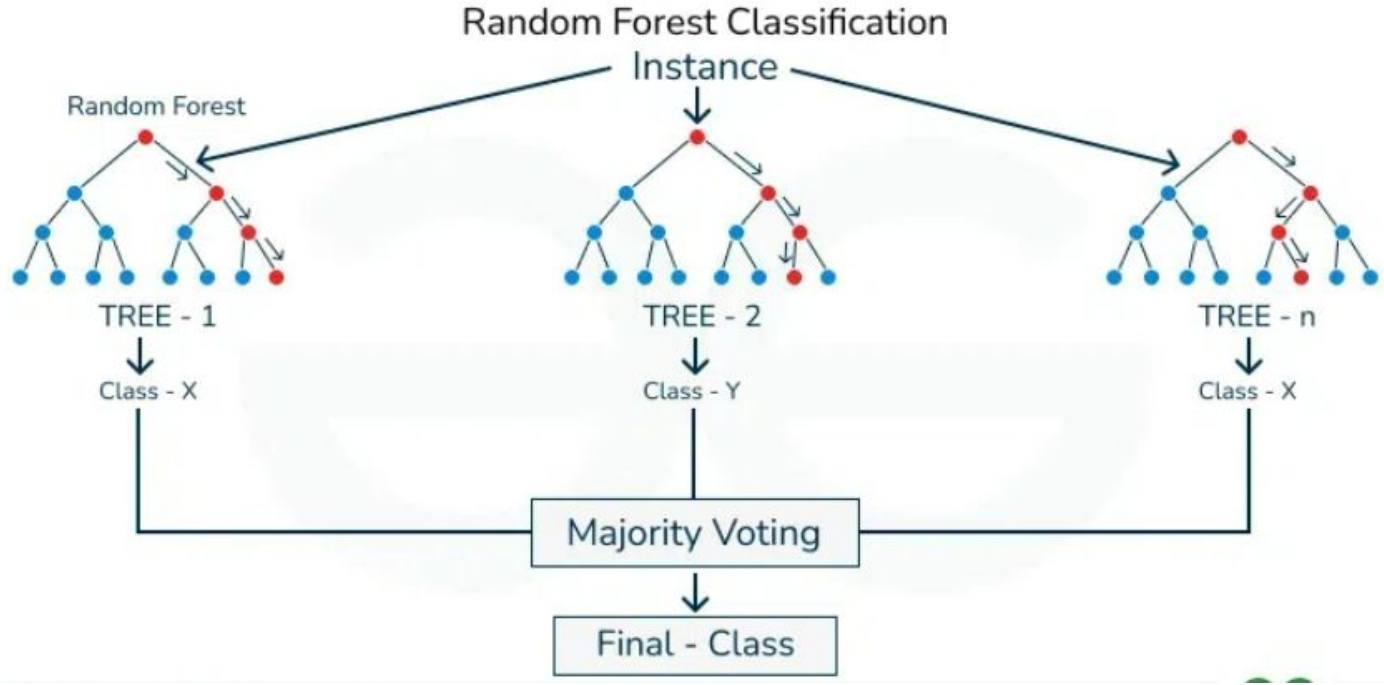
- **Problématique:** Identifier les clients susceptibles de résilier leur abonnement à un service.
- **Données:** Historique d'achat, fréquence d'utilisation, satisfaction client, etc.
- **Utilisation:** Les forêts aléatoires peuvent prédire avec une bonne précision les clients à risque de churn, permettant aux entreprises de mettre en place des actions de fidélisation ciblées.

### 2. Classification d'Images

- **Problématique:** Identifier le contenu d'une image (animal, objet, personne).
- **Données:** Pixels de l'image.
- **Utilisation:** Bien que les réseaux de neurones convolutifs soient plus couramment utilisés pour la vision par ordinateur, les forêts aléatoires peuvent également être employées, en particulier pour des jeux de données plus petits ou des problèmes moins complexes.

# FORÊT ALÉATOIRE- RANDOM FOREST

## Diagramme:



# FORÊT ALÉATOIRE- RANDOM FOREST

## Implémentation en Python avec scikit-learn:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import
 accuracy_score

# Charger les données
X = ...  # Variables explicatives
y = ...  # Variable cible

# Diviser les données en ensembles d'entraînement et de test
X_train, X_test, y_train, y_test = train_test_split(X, y,
 test_size=0.2, random_state=42)


# Créer et entraîner la forêt aléatoire
model = RandomForestClassifier(n_estimators=100)  # 100 arbres
model.fit(X_train, y_train)

# Faire des prédictions
y_pred = model.predict(X_test)

# Évaluer le modèle
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

# FORÊT ALÉATOIRE- RANDOM FOREST

## Implémentation en Python avec scikit-learn:

**Hyperparamètres clés**

- **n_estimators:** Nombre d'arbres dans la forêt.
- **max_depth:** Profondeur maximale de chaque arbre.
- **min_samples_split:** Nombre minimum d'échantillons requis pour diviser un nœud.
- **min_samples_leaf:** Nombre minimum d'échantillons requis à chaque feuille.
- **criterion:** Critère utilisé pour la sélection des attributs (e.g., 'gini', 'entropy').

## Visualisation de Forêt Aléatoire:

```
from sklearn.tree import plot_tree
plt.figure(figsize=(20,10))
plot_tree(model.estimators_[0], filled=True,
feature_names=X.columns, class_names=['0', '1'])
plt.show()
```

# FORÊT ALÉATOIRE- RANDOM FOREST

## Visualisation de Forêt Aléatoire:

Pour une visualisation plus détaillée et personnalisable, on peut utiliser **Graphviz**.

```
from sklearn.tree import export_graphviz
import graphviz

dot_data = export_graphviz(model.estimators_[0], out_file=None,
                           feature_names=X.columns,
                           class_names=['0', '1'],
                           filled=True, rounded=True,
                           special_characters=True)
graph = graphviz.Source(dot_data)
graph.render("iris")
```

# SUPPORT VECTOR MACHINE SVM

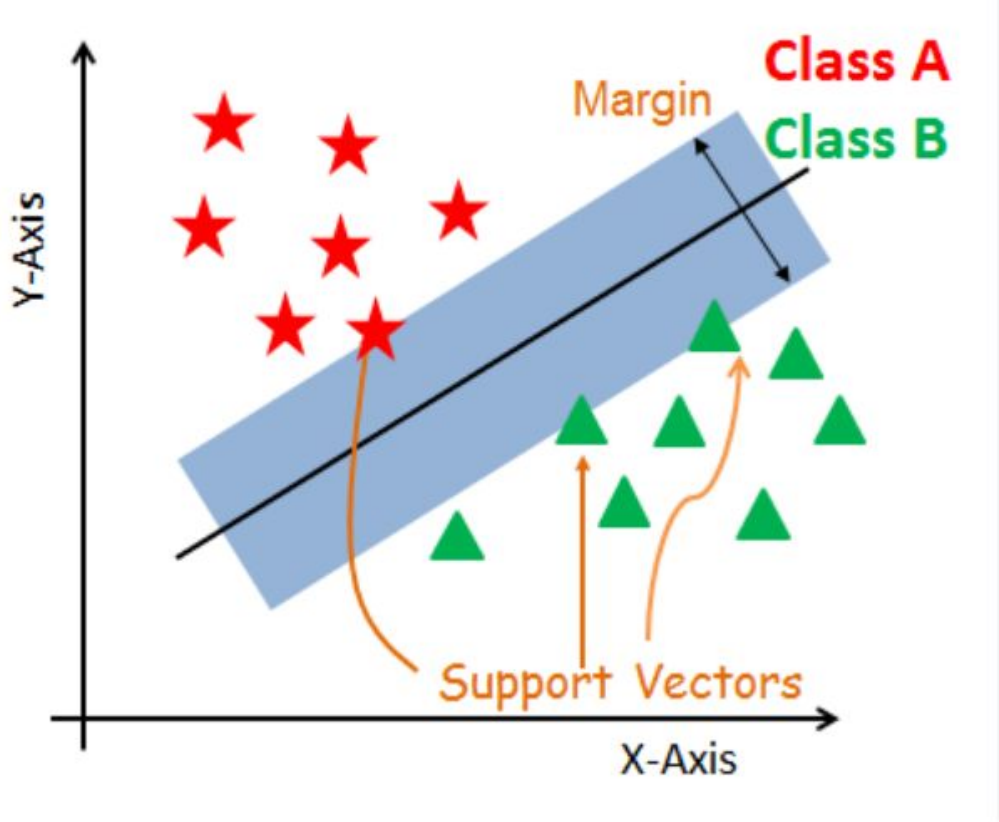
- **Machine à Vecteurs de Support,** est un algorithme d'apprentissage supervisé.
- Son but principal est de classer des données en différentes catégories. Imaginez que vous ayez des points de différentes couleurs sur un graphique. Une SVM va essayer de trouver la meilleure ligne (ou hyperplan en dimension supérieure) pour séparer ces points de couleurs différentes.

## Comment ça marche ?

- **Hyperplan:** C'est la ligne (ou le plan) qui sépare au mieux les données en différentes classes.
- **Marge:** C'est la distance entre l'hyperplan et les points de données les plus proches, appelés **vecteurs de support**.
- **Optimisation:** L'algorithme SVM cherche à trouver l'hyperplan qui maximise cette marge. Plus la marge est grande, meilleure est la généralisation du modèle.

# SUPPORT VECTOR MACHINE SVM

"Dans le cas le plus simple, imaginons que nous avons des données qui peuvent être séparées par une ligne droite. Cette ligne est notre hyperplan. L'objectif est de trouver la ligne qui est le plus éloignée des points de données des deux classes. Cette distance, c'est ce qu'on appelle la marge. Plus la marge est grande, meilleure est la généralisation de notre modèle."



# SUPPORT VECTOR MACHINE SVM

## Le Noyau (Kernel)

Le noyau est un concept fondamental dans les Machines à Vecteurs de Support (SVM). Il joue un rôle crucial en permettant aux SVM de gérer des problèmes de classification non linéaires.

- **Limite des SVM linéaires:** Les SVM de base sont conçues pour trouver un hyperplan qui sépare linéairement les données. Or, de nombreux problèmes réels ne sont pas linéairement séparables.
- **Le rôle du noyau:** Le noyau permet de projeter les données dans un espace de plus haute dimension où elles peuvent devenir linéairement séparables. Cette projection est souvent implicite et calculée efficacement grâce à une astuce mathématique.

# SUPPORT VECTOR MACHINE SVM

## Le Noyau (Kernel)

### Comment ça marche ?

1. **Espace de caractéristiques:** Le noyau définit un espace de caractéristiques implicite, souvent de dimension infinie.
2. **Produit scalaire:** Au lieu de calculer explicitement les coordonnées des données dans cet espace de haute dimension, le noyau calcule directement le produit scalaire entre deux points dans cet espace.
3. **Astuce du noyau:** Cette astuce permet d'éviter les calculs coûteux associés à la projection dans un espace de haute dimension.

**En résumé,** le noyau transforme l'espace initial en un espace où les données peuvent être séparées linéairement, même si elles ne l'étaient pas dans l'espace original.

# SUPPORT VECTOR MACHINE SVM

## Le Noyau (Kernel)

### Types de noyaux couramment utilisés :

- **Noyau linéaire:** Le plus simple, il correspond à un produit scalaire standard. Il est utilisé lorsque les données sont linéairement séparables.
- **Noyau polynomial:** Permet de capturer des relations polynomiales entre les données.
- **Noyau RBF (Radial Basis Function):** Très populaire, il permet de capturer des relations non linéaires complexes.
- **Autres noyaux:** Sigmoïde, etc.

### Choix du noyau :

Le choix du noyau dépend du problème à résoudre et des caractéristiques des données. Il n'existe pas de règle universelle. On utilise souvent une approche empirique en testant différents noyaux et en sélectionnant celui qui donne les meilleurs résultats.

# SUPPORT VECTOR MACHINE SVM

## Exemples Concrets:

### 1. Reconnaissance d'Images

- **Principe:** Une image est transformée en un vecteur numérique (par exemple, en utilisant des matrices de pixels). Une SVM est entraînée à classifier ces vecteurs en différentes catégories (visages, voitures, animaux, etc.).
- **Avantages:** Les SVM peuvent gérer des ensembles de données d'images complexes et identifier des caractéristiques visuelles subtiles.

### 2. Bioinformatique

- **Principe:** Les séquences d'ADN sont représentées sous forme de vecteurs numériques. Une SVM peut être utilisée pour prédire si une séquence code pour une protéine, pour identifier des sites de liaison pour des protéines, ou pour classer les gènes en fonction de leur fonction.
- **Avantages:** Les SVM sont efficaces pour traiter de grandes quantités de données biologiques et identifier des motifs complexes.

### 3. Analyse de Sentiment

- **Principe:** Un texte (tweet, commentaire, avis) est converti en un vecteur numérique représentant la présence de mots clés positifs ou négatifs. Une SVM est entraînée à prédire le sentiment global du texte (positif, négatif, neutre).
- **Avantages:** Les SVM peuvent capturer les nuances du langage naturel et identifier les émotions exprimées dans un texte.

# SUPPORT VECTOR MACHINE SVM

## Exemples Concrets:

### 4. Finance

- **Principe:** Les données financières (cours de bourse, indicateurs économiques) sont utilisées pour entraîner une SVM à prédire l'évolution future des marchés.
- **Avantages:** Les SVM peuvent capturer des tendances complexes et non-linéaires dans les données financières.

### 5. Détection de Spam

- **Principe:** Chaque email est représenté comme un vecteur de mots (ou n-grammes). Une SVM est entraînée à distinguer les emails "spam" des emails "non-spam" en se basant sur la fréquence d'apparition de ces mots.
- **Avantages:** Les SVM sont très efficaces pour capturer les subtilités linguistiques et les modèles complexes qui caractérisent les spams.

# SUPPORT VECTOR MACHINE SVM

## Implémentation de SVM en Python: Code

```
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score

# Charger le jeu de données Iris
iris = load_iris()
X = iris.data
y = iris.target

# Diviser les données en ensembles d'entraînement et de test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,

# Créer un modèle SVM avec un noyau RBF (Radial Basis Function)
model = SVC(kernel='rbf', gamma='scale')

# Entraîner le modèle
model.fit(X_train, y_train)

# Faire des prédictions sur l'ensemble de test
y_pred = model.predict(X_test)

# Évaluer les performances du modèle
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

# SUPPORT VECTOR MACHINE SVM

## Implémentation de SVM en Python: Explication détaillée

**-Importation des modules:** On importe les modules nécessaires de scikit-learn pour travailler avec les SVM, diviser les données, charger les données et évaluer les performances.

**-Chargement des données:** On charge le jeu de données Iris, qui contient des informations sur différentes espèces d'iris.

**-Division des données:** On divise les données en deux ensembles : un ensemble d'entraînement pour entraîner le modèle et un ensemble de test pour évaluer ses performances.

**-Création du modèle:** On crée un modèle SVM avec un noyau RBF, qui est un choix courant pour les problèmes non linéaires. Le paramètre `gamma='scale'` permet d'ajuster automatiquement le paramètre gamma du noyau.

**-Entraînement du modèle:** On entraîne le modèle sur l'ensemble d'entraînement.

**-Prédictions:** On utilise le modèle entraîné pour faire des prédictions sur l'ensemble de test.

**-Évaluation:** On calcule la précision du modèle en comparant les prédictions aux vraies étiquettes.

# SUPPORT VECTOR MACHINE SVM

## Implémentation de SVM en Python: Personnalisation du modèle

**Choix du noyau:** Vous pouvez expérimenter avec différents noyaux (linear, poly, sigmoid) en fonction de votre problème.

**Réglage des hyperparamètres:** Les paramètres `C` et `gamma` contrôlent la complexité du modèle et peuvent être ajustés pour améliorer les performances.

**Autres métriques:** Vous pouvez utiliser d'autres métriques d'évaluation comme la précision, le rappel, la F1-score, etc. pour évaluer votre modèle de manière plus approfondie.

## Recherche par Grille: GRID SEARCH

La recherche par grille est une technique d'optimisation utilisée en apprentissage automatique supervisé pour trouver les meilleures valeurs des hyperparamètres d'un modèle. Un hyperparamètre est un paramètre dont la valeur est fixée avant l'entraînement du modèle et qui influence directement ses performances.

**Comment ça marche ?**

1. **Définition d'une grille:** On définit un ensemble de valeurs possibles pour chaque hyperparamètre à optimiser. Cela crée une grille de toutes les combinaisons possibles.

# SUPPORT VECTOR MACHINE SVM

2. **Entraînement du modèle pour chaque combinaison:** Pour chaque combinaison des hyperparamètres, un modèle est entraîné sur les données d'entraînement et évalué sur les données de validation.

3. **Sélection de la meilleure combinaison:** La combinaison des hyperparamètres qui donne les meilleures performances sur les données de validation est sélectionnée.

**Exemple avec Grid Search:** Pour trouver les meilleurs hyperparamètres, vous pouvez utiliser la recherche par grille:

```
from sklearn.model_selection import GridSearchCV

param_grid = {'C': [0.1, 1, 10], 'gamma': [0.01, 0.1, 1]}
grid = GridSearchCV(SVC(), param_grid, refit=True, verbose=3)
grid.fit(X_train, y_train)
print(grid.best_params_)
```

# SUPPORT VECTOR MACHINE SVM

## Exemple avec Grid Search:

```
from sklearn.model_selection import GridSearchCV
from sklearn.svm import SVC

# Définition des hyperparamètres à tester
param_grid = {'C': [0.1, 1, 10], 'kernel': ['linear', 'rbf']}

# Création du modèle SVM
svc = SVC()

# Recherche par grille
grid = GridSearchCV(svc, param_grid, refit=True, verbose=3)
grid.fit(X_train, y_train)

# Meilleurs paramètres
print(grid.best_params_)
```

# SUPPORT VECTOR MACHINE SVM

## Exemple avec Grid Search:

`-param_grid` définit une grille avec deux hyperparamètres : `C` (pénalisation) et `kernel` (noyau).

`-GridSearchCV` va entraîner le modèle SVM pour chaque combinaison de ces paramètres et sélectionner la meilleure.

`-refit=True` permet de réentraîner le modèle avec les meilleurs paramètres sur l'ensemble des données d'entraînement.

# SUPPORT VECTOR MACHINE SVM

## Le scaling des données:

Le scaling, ou mise à l'échelle consiste à transformer les données numériques pour qu'elles soient sur une échelle commune. Cela est particulièrement important lorsque les variables d'un dataset ont des échelles très différentes. Par exemple, une variable mesurée en mètres et une autre en milliers d'euros ne sont pas directement comparables si elles ne sont pas mises à la même échelle.

## Pourquoi scaler les données ?

- Convergence plus rapide des algorithmes d'optimisation
- Amélioration des performances des modèles
- Importance égale de toutes les caractéristiques

***Il est généralement recommandé de scaler les données après avoir divisé les données en ensembles d'entraînement et de test***

# SUPPORT VECTOR MACHINE SVM

## Comment scaler les données ?

- **Standardisation:** Centrage et réduction à l'écart-type unitaire. Chaque valeur est transformée en un score z.
- **Normalisation:** Mise à l'échelle des valeurs entre 0 et 1.
- **MinMaxScaler:** Similaire à la normalisation, mais avec des bornes personnalisables.
- **RobustScaler:** Moins sensible aux valeurs aberrantes que la standardisation.

## Scaling avec Python

```
from sklearn.preprocessing import StandardScaler

# Créer un scaler
scaler = StandardScaler()

# Ajuster le scaler sur les données d'entraînement
scaler.fit(X_train)

# Transformer les données d'entraînement et de test
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

# SUPPORT VECTOR MACHINE SVM

**Exemple de Classification avec SVM**

**(Iris dataset)**

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC

# Charger un dataset (exemple : dataset Iris)
from sklearn.datasets import load_iris
X, y = load_iris(return_X_y=True)

# Diviser les données en ensembles d'entraînement et de test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,

# Créer un scaler (ici, StandardScaler)
scaler = StandardScaler()

# Ajuster le scaler sur les données d'entraînement
scaler.fit(X_train)

# Transformer les données d'entraînement et de test
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Créer un modèle SVM
model = SVC()

# Entraîner le modèle sur les données scalées
model.fit(X_train_scaled, y_train)

# Faire des prédictions sur les données de test scalées
y_pred = model.predict(X_test_scaled)

# Évaluer les performances du modèle
# ... (utiliser des métriques comme accuracy, precision, recall, F1-score
```

# UNSUPERVISED MACHINE LEARNING
# APPRENTISSAGE NON SUPERVISÉ

# INTRODUCTION

## Définition:

- L'apprentissage non supervisé est une branche du machine learning où les algorithmes apprennent à partir de données non étiquetées.
- l'algorithme doit identifier lui-même les structures, les patterns ou les groupes cachés dans les données.
- L'objectif principal est de découvrir des informations intrinsèques aux données sans avoir besoin d'un superviseur.

## Pourquoi utiliser l'apprentissage non supervisé ?

- Découverte de structures cachées: Identifier des groupes naturels dans les données (clustering), détecter des anomalies (détection de fraude), réduire la dimensionnalité des données (ACP).

# INTRODUCTION

## Pourquoi utiliser l'apprentissage non supervisé ?

- Découverte de structures cachées: Identifier des groupes naturels dans les données (clustering), détecter des anomalies (détection de fraude), réduire la dimensionnalité des données (ACP).
- Préparation des données: Prétraitement des données pour les rendre plus adaptées à d'autres algorithmes (ex : réduction de dimensionnalité avant classification).
- Génération de nouvelles données: Les réseaux de neurones génératifs (GANs) peuvent créer de nouvelles images, de nouvelles musiques, etc.

# INTRODUCTION

## Les principaux algorithmes d'apprentissage non supervisé:

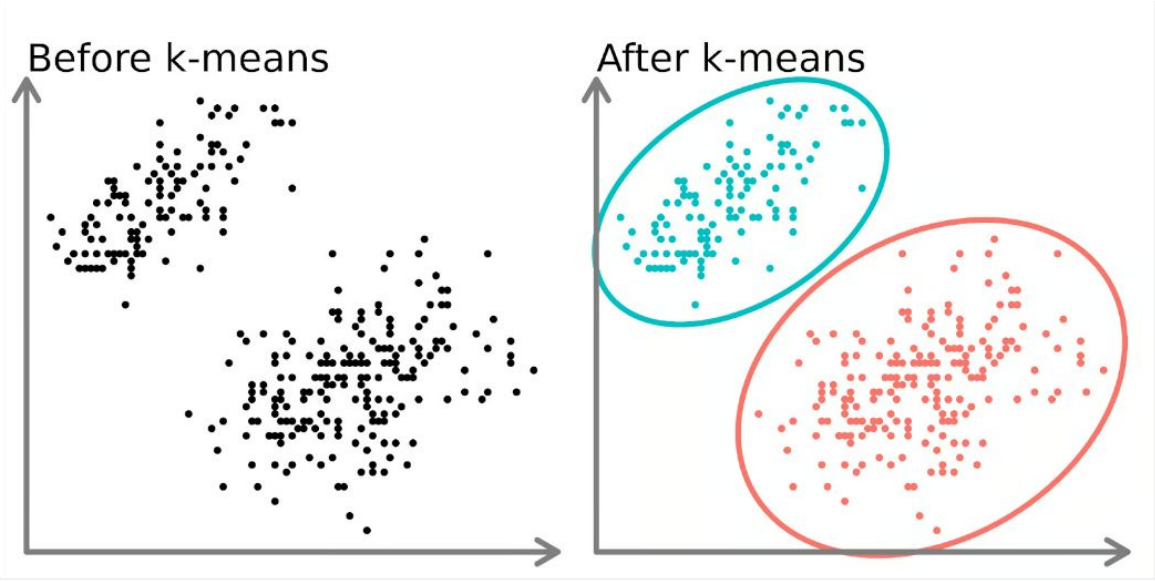
Nous allons voir les  algorithmes de ***Clustering***:

- **K-means:** Divise les données en K groupes.
- **DBSCAN:** Identifie des clusters de densité variable.
- **Hierarchical clustering:** Crée une hiérarchie de clusters.

# K-MEANS

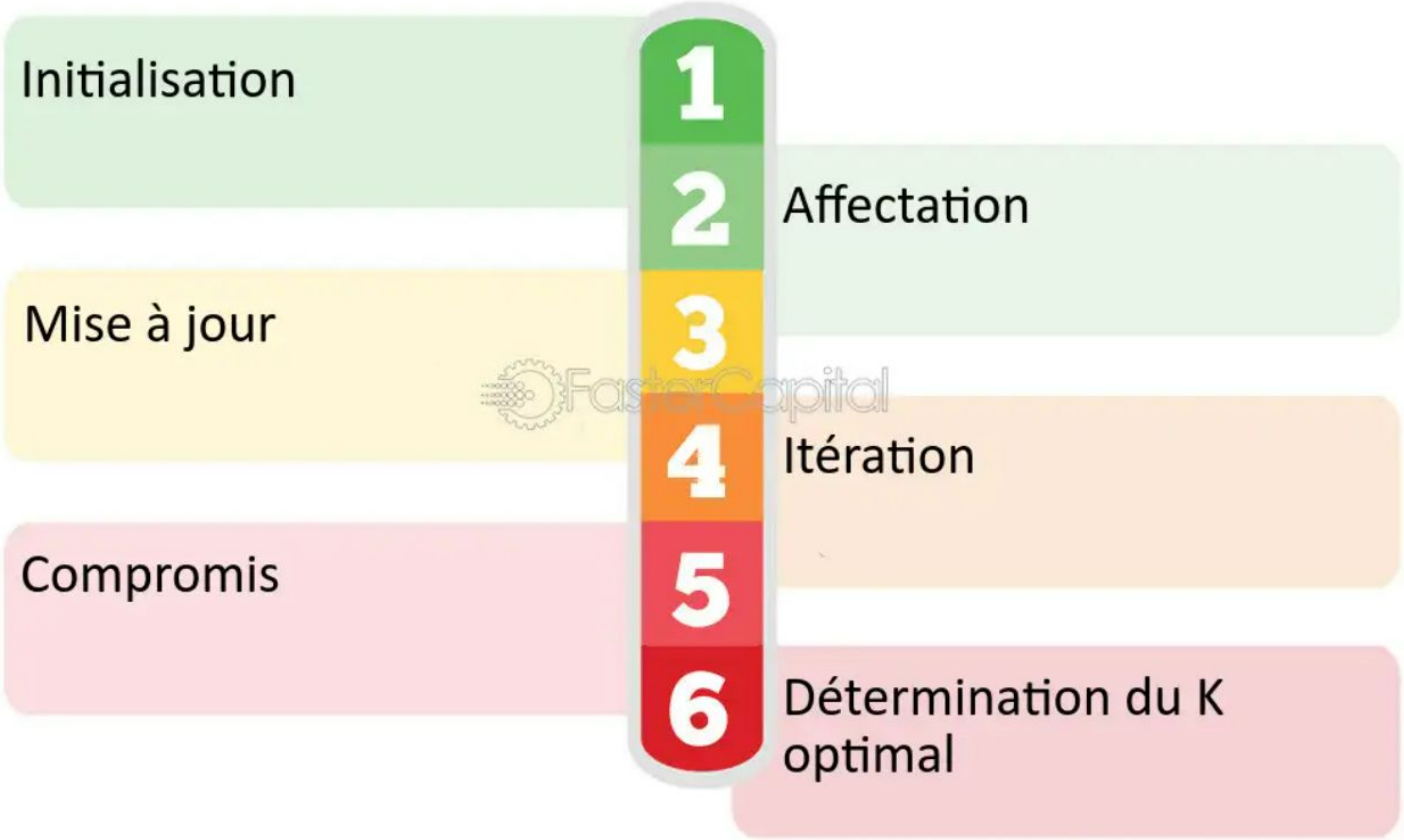
## Définition

Il vise à partitionner un ensemble de données en *K* groupes (ou clusters) distincts. L'objectif est de former des groupes de manière à ce que les points à l'intérieur d'un même groupe soient les plus similaires possible, tandis que les points de groupes différents soient les plus différents possible.



# K-MEANS

## Processus de l'algorithme K-means



# K-MEANS

## Fonctionnement

1. **Initialisation:**

On choisit aléatoirement *K* points dans l'espace des données comme centres de clusters initiaux.

2. **Affectation:**

Chaque point de données est assigné au cluster dont le centre est le plus proche (généralement en utilisant la distance euclidienne).

3. **Calcul des nouveaux centres:**

Pour chaque cluster, on calcule la nouvelle position du centre en prenant la moyenne de tous les points qui lui sont assignés.

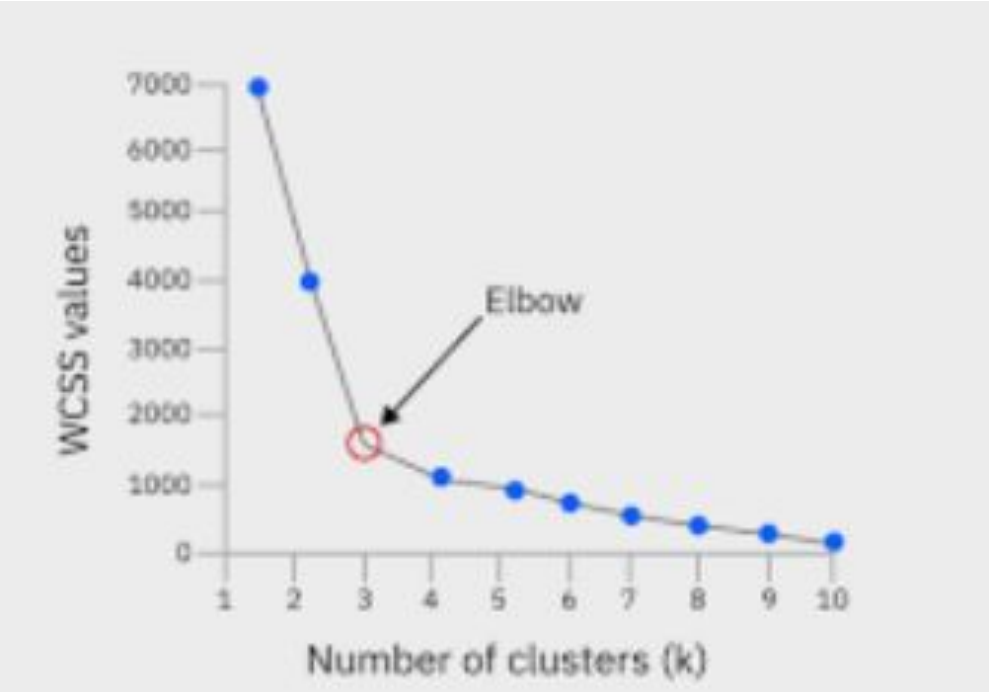
4. **Itération:**

On répète les étapes 2 et 3 jusqu'à ce que les centres des clusters ne bougent plus significativement (convergence) ou jusqu'à ce qu'un nombre maximum d'itérations soit atteint.

# K-MEANS

## Choix de K

**Méthode du coude:** On trace l'inertie (somme des distances au carré entre chaque point et le centre de son cluster) en fonction de K. Le coude de la courbe correspond généralement à un bon choix de K.



# K-MEANS

## Choix de K: Méthode du coude: Code Python

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

# Génération de données aléatoires
X, y_true = make_blobs(n_samples=300, centers=4,
                       cluster_std=0.60, random_state=0)

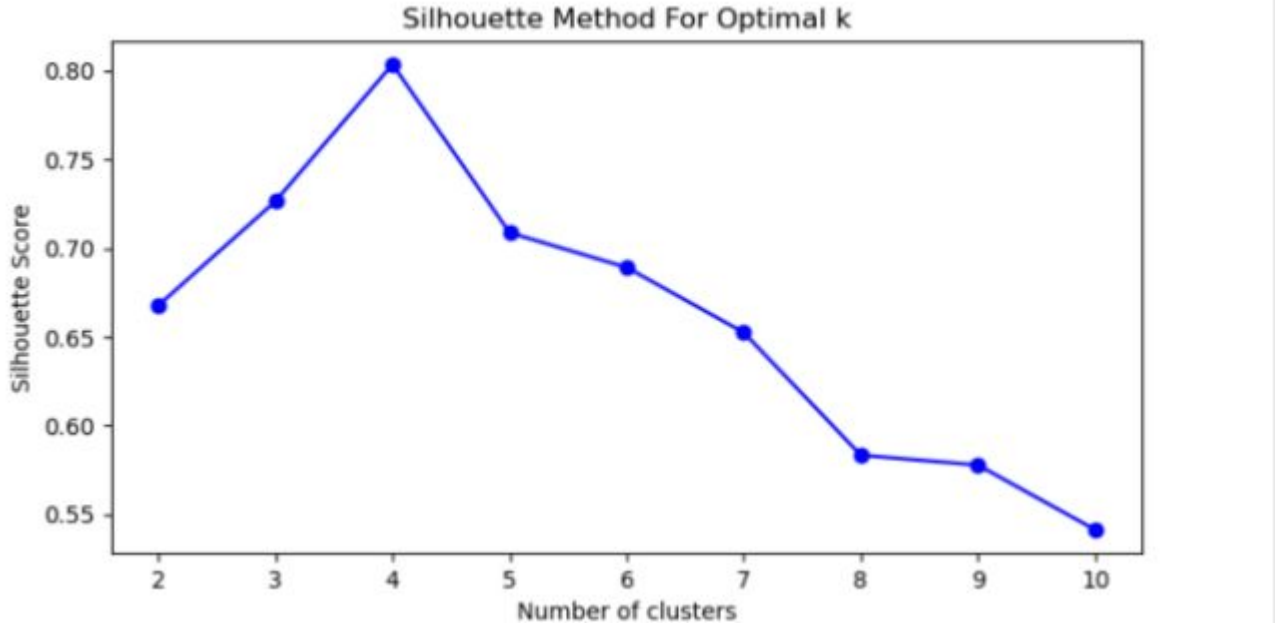
# Calcul de l'inertie pour différents nombres de clusters
wcss = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, init='k-means++', max_iter=300, n_init=10, random_state=0)
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)

# Tracer la courbe de l'inertie
plt.plot(range(1, 11), wcss)
plt.title('Elbow Method')
plt.xlabel('Number of clusters')
plt.ylabel('WCSS') # Within cluster sum of squares
plt.show()
```

# K-MEANS

## Choix de K

**Silhouette score:** Mesure la qualité de l'affectation d'un point à un cluster en comparant sa distance aux autres points de son cluster et à ceux des clusters voisins.



# K-MEANS

## Choix de K Silhouette score: Code Python

```python
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# Données d'exemple (remplacez par vos propres données)
X = ...  # Votre matrice de données

# Création d'un modèle K-means avec un nombre spécifique de clusters
kmeans = KMeans(n_clusters=3, random_state=0)

# Entraînement du modèle sur les données
kmeans.fit(X)

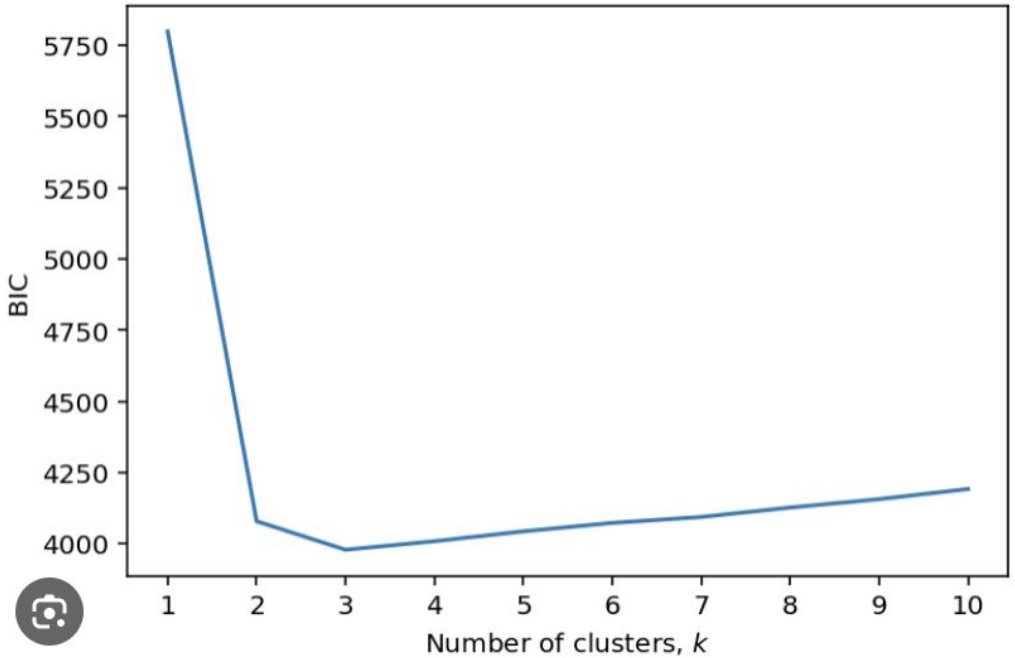
# Prédiction des étiquettes de cluster pour chaque point de données
labels = kmeans.labels_

# Calcul du score de silhouette
silhouette_avg = silhouette_score(X, labels)
print("Score de silhouette :", silhouette_avg)
```

# K-MEANS

## Choix de K

**Critères d'information:** Comme le **critère de Bayes information (BIC)**, On choisit le modèle qui minimise le critère



# K-MEANS

**Choix de K :** **Critères d'information:Code Python**

Formule du BIC pour K-means:

```
BIC = k * ln(n) - 2 * ln(L)
```

- **k:** Nombre de paramètres du modèle (ici, le nombre de centres de clusters multiplié par la dimension des données)
- **n:** Nombre d'observations
- **L:** Vraisemblance maximale du modèle

```python
import numpy as np
from sklearn.cluster import KMeans

def calculer_bic(kmeans, X):
    """
    Calcule le Critère d'Information Bayésien (BIC) pour un modèle KMeans

    Args:
        kmeans: Un modèle KMeans ajusté.
        X: Les données.

    Returns:
        La valeur du BIC.
    """
    # k: nombre de clusters
    k = kmeans.n_clusters
    # n: nombre de points de données
    n = len(X)
    # log-vraisemblance (équivalent à l'opposé de l'inertie dans KMeans)
    logL = kmeans.score(X)
    bic = k * np.log(n) - 2 * logL
    return bic

# Données d'exemple (remplacer par vos propres données)
X = ...

# Tester différents nombres de clusters
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=0).fit(X)
    bic = calculer_bic(kmeans, X)
    print(f"Pour k={k} clusters, le BIC est : {bic}")
```

# K-MEANS

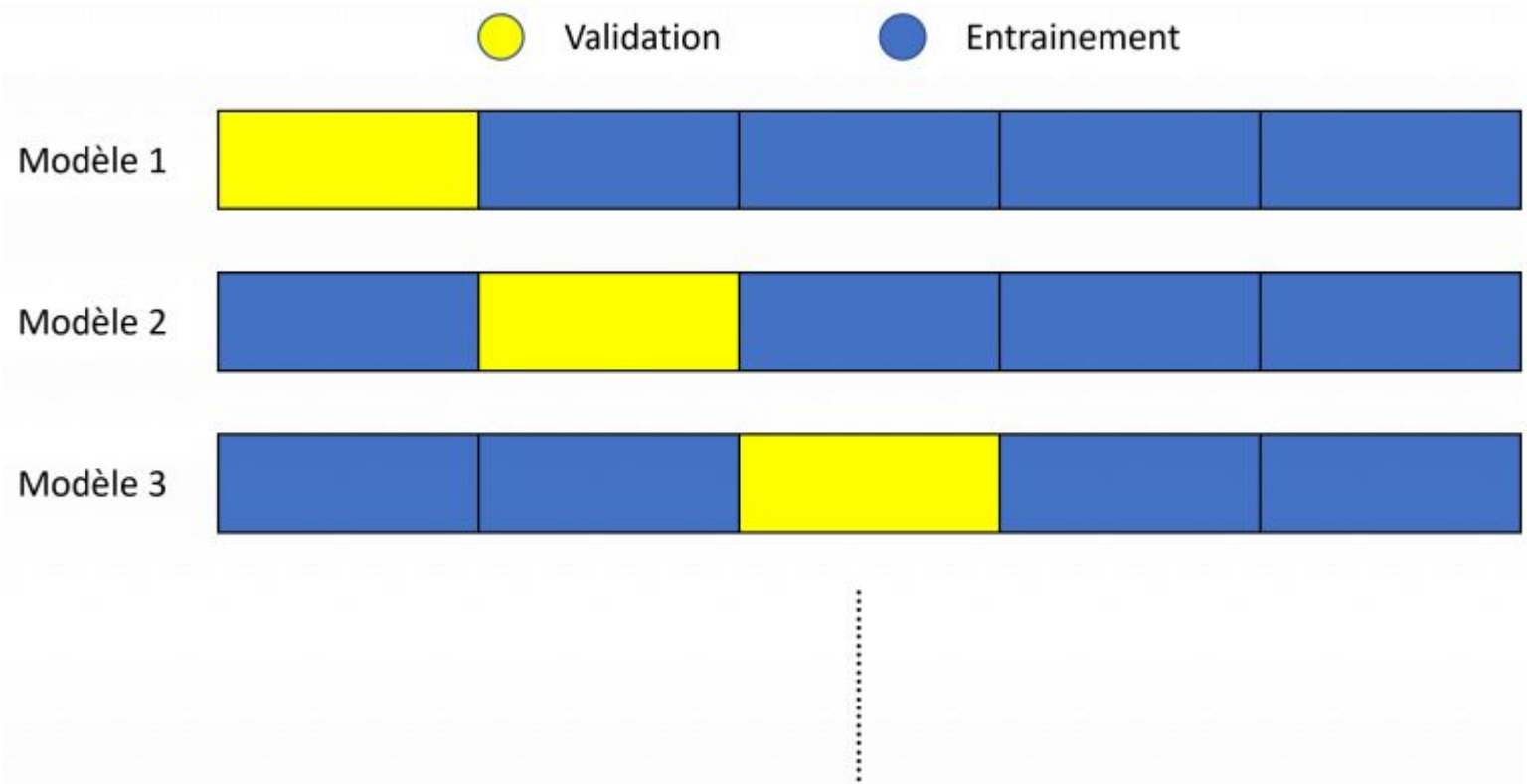
## Choix de K:

**Validation croisée:**

La validation croisée K-fold est une méthode très utilisée. Elle consiste à diviser le jeu de données en K sous-ensembles de taille similaire (folds). À chaque itération, un fold est utilisé comme ensemble de test, tandis que les K-1 autres folds sont utilisés pour l'entraînement. On obtient ainsi K évaluations des performances du modèle, que l'on peut moyenner pour obtenir une estimation plus fiable.

# K-MEANS

**Choix de K:** **Validation croisée:**



```python
from sklearn.model_selection import KFold
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Données d'exemple
X = ...  # Matrice des features
y = ...  # Vecteur des targets

# Nombre de folds
k = 5

# Créer un objet KFold
kf = KFold(n_splits=k, shuffle=True, random_state=42)

# Liste pour stocker les scores
scores = []

# Boucle sur chaque fold
for train_index, test_index in kf.split(X):
    X_train, X_test = X[train_index], X[test_index]
    y_train, y_test = y[train_index], y[test_index]

    # Entraîner le modèle sur le fold d'entraînement
    model = LinearRegression()
    model.fit(X_train, y_train)

    # Évaluer le modèle sur le fold de test
    y_pred = model.predict(X_test)
    score = mean_squared_error(y_test, y_pred)
    scores.append(score)

# Calcul du score moyen
mean_score = np.mean(scores)
print("Score moyen de la validation croisée :", mean_score)
```

# K-MEANS

## Choix de K: Validation croisée:

### Explication du code :

1. **Importation des bibliothèques:** `sklearn.model_selection` pour la validation croisée, `sklearn.linear_model` pour le modèle de régression linéaire et `sklearn.metrics` pour l'évaluation.
2. **Création des données:** Remplacer `X` et `y` par vos propres données.
3. **Création d'un objet KFold:** `n_splits` spécifie le nombre de folds, `shuffle` indique si les données doivent être mélangées avant la division, et `random_state` fixe la graine aléatoire pour la reproductibilité.
4. **Boucle sur les folds:**
   - Division des données en ensembles d'entraînement et de test.
   - Entraînement du modèle sur l'ensemble d'entraînement.
   - Évaluation du modèle sur l'ensemble de test et stockage du score.
5. **Calcul du score moyen:** Calcul de la moyenne des scores obtenus sur tous les folds.

# K-MEANS

## Implémentation en Python avec Scikit-learn

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

# Génération de données aléatoires pour l'exemple
X, y_true = make_blobs(n_samples=300, centers=4,
                       cluster_std=0.60, random_state=0)

# Affichage des données
plt.scatter(X[:, 0], X[:, 1], s=50)
plt.show()

# Création du modèle K-means avec 4 clusters
kmeans = KMeans(n_clusters=4, random_state=0)

# Entraînement du modèle
kmeans.fit(X)

# Prédictions (étiquettes des clusters)
y_kmeans = kmeans.predict(X)

# Visualisation des résultats
plt.scatter(X[:, 0], X[:, 1], c=y_kmeans, s=50, cmap='viridis')

centers = kmeans.cluster_centers_
plt.scatter(centers[:, 0], centers[:, 1], c='black', s=200, alpha=0.5)
plt.show()
```

# K-MEANS

## Explication détaillée du code

1. **Génération de données:** On utilise `make_blobs` pour créer un dataset artificiel avec 4 clusters.
2. **Visualisation:** On affiche les données pour avoir une idée de leur distribution.
3. **Création du modèle:** On initialise un modèle K-means avec 4 clusters.
4. **Nombre de clusters (n_clusters):** Ce paramètre est crucial. Il peut être déterminé à l'aide de méthodes comme la méthode du coude ou le silhouette score.
5. **Initialisation:** K-means est sensible à l'initialisation des centres. `KMeans++` est une méthode d'initialisation plus robuste.
6. **Métrique de distance:** Par défaut, K-means utilise la distance euclidienne. D'autres métriques peuvent être utilisées en fonction du problème.
7. **Nombre d'itérations:** Le paramètre `max_iter` contrôle le nombre maximum d'itérations.
8. **Critère d'arrêt:** Le paramètre `tol` définit la tolérance pour la convergence.

# K-MEANS

## Implémentation 2 en Python avec Scikit-learn

```
import numpy as np
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.model_selection import
cross_val_score
import matplotlib.pyplot as plt

# Charger vos données (exemple avec un dataset
de scikit-learn)
from sklearn.datasets import load_iris
iris = load_iris()
X = iris.data

# Liste des valeurs de k à tester
k_range = range(1, 11)

# Liste pour stocker les scores moyens (indice de
silhouette)
silhouette_scores = []
```

```
# Boucle pour chaque valeur de k
for k in k_range:
    km = KMeans(n_clusters=k, random_state=42)
    scores = cross_val_score(km, X, cv=5,
scoring='silhouette')
    silhouette_scores.append(np.mean(scores))

# Tracer le graphique de l'indice de silhouette
plt.plot(k_range, silhouette_scores)
plt.xlabel('Nombre de clusters (k)')
plt.ylabel('Score de silhouette moyen')
plt.title('Choix du nombre optimal de clusters')
plt.show()

# Identifier le k optimal (celui avec le score de
silhouette maximal)
optimal_k = np.argmax(silhouette_scores) + 1
print("Le nombre optimal de clusters est :", optimal_k)
```

# K-MEANS

## Implémentation 2 en Python avec Scikit-learn

**Explication détaillée :**

1. **Importation des bibliothèques:** On importe les bibliothèques nécessaires pour la manipulation de données (NumPy, Pandas), le clustering (KMeans), la validation croisée (cross_val_score) et la visualisation (Matplotlib).
2. **Chargement des données:** On charge un exemple de dataset (iris) pour illustrer le code. Remplacez ceci par votre propre dataset.
3. **Définition des valeurs de k:** On crée une liste des valeurs de k à tester.
4. **Calcul des scores de silhouette:**
    - Pour chaque valeur de k :
        - On crée un modèle KMeans.
        - On utilise `cross_val_score` pour évaluer le modèle avec une validation croisée à 5 folds. On utilise le score de silhouette comme métrique d'évaluation.
        - On calcule la moyenne des scores de silhouette pour chaque valeur de k.
5. **Visualisation:** On trace un graphique pour visualiser l'évolution du score de silhouette en fonction du nombre de clusters.
6. **Identification du k optimal:** On détermine la valeur de k qui correspond au score de silhouette maximal.

# K-MEANS

## Applications pratiques de K-means

- **Segmentation de clientèle:** Regrouper les clients en fonction de leurs comportements d'achat.
- **Analyse d'images:** Segmenter des images en régions homogènes.
- **Détection d'anomalies:** Identifier des points de données qui ne correspondent à aucun cluster.
- **Compression de données:** Réduire la dimensionnalité des données en représentant chaque point par son centre de cluster.

# K-MEANS

## Limitations de K-means

- **Forme des clusters:** K-means suppose des clusters de forme convexe et de densité similaire.
- **Nombre de clusters:** Il est nécessaire de connaître à l'avance le nombre de clusters.
- **Sensibilité aux outliers:** Les outliers peuvent influencer significativement les centres de clusters.

# DBSCAN

# Density-Based Spatial Clustering of Applications with Noise

# DBSCAN

## Définition:

C’est un algorithme de clustering qui identifie les clusters en fonction de la densité des données. Contrairement à des algorithmes comme K-means qui requièrent de spécifier à l'avance le nombre de clusters

DBSCAN est capable de découvrir des clusters de formes arbitraires et de gérer les données bruitées.

## Principe de fonctionnement:

DBSCAN repose sur deux paramètres clés :

**Eps (ε):** Le rayon d'un voisinage.

**MinPts:** Le nombre minimum de points requis dans un voisinage pour qu'un point soit considéré comme un point de cœur.

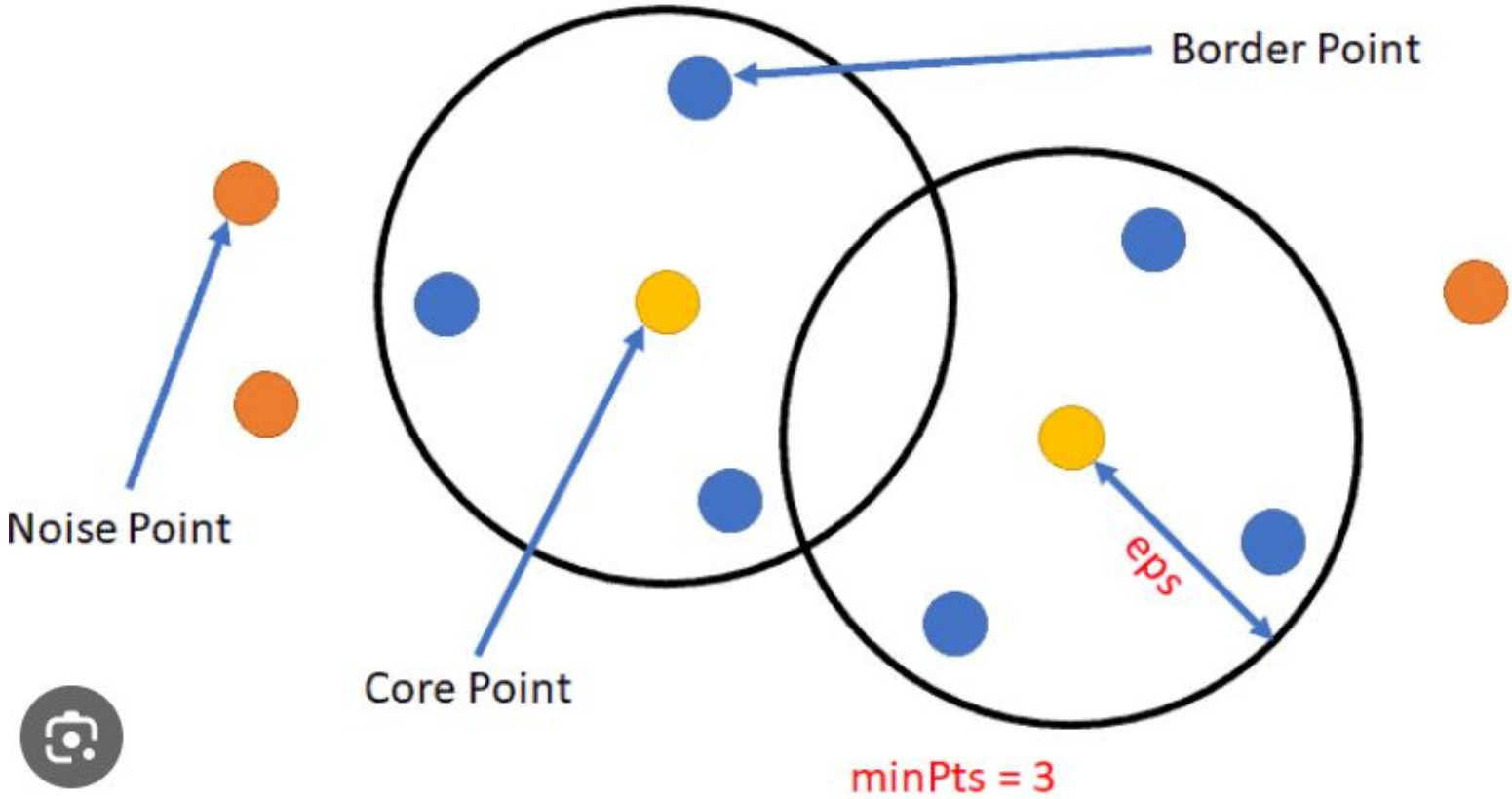
# DBSCAN

## Principe de fonctionnement:

L'algorithme fonctionne de la manière suivante :

1. **Choix d'un point au hasard:** Un point est sélectionné aléatoirement.
2. **ε-voisinage:** On détermine tous les points situés à une distance inférieure à ε du point choisi.
3. **Point de cœur:** Si le nombre de points dans l'ε-voisinage est supérieur ou égal à MinPts, le point est considéré comme un point de cœur.
4. **Formation du cluster:** A partir du point de cœur, on explore récursivement les ε-voisinages des points voisins pour former un cluster.
5. **Points bruits:** Les points qui ne sont ni des points de cœur ni atteignables depuis un point de cœur sont considérés comme des bruits.

# DBSCAN



# DBSCAN

## Implémentation en Python avec Scikit-learn de DBSCAN

### Choix des paramètres ε et MinPts

- **ε:** Une valeur trop petite peut conduire à la création de nombreux petits clusters, tandis qu'une valeur trop grande peut fusionner des clusters distincts.
- **MinPts:** Un MinPts élevé peut conduire à la création de moins de clusters, tandis qu'un MinPts faible peut conduire à la création de nombreux petits clusters.

La détermination des valeurs optimales pour ε et MinPts peut être réalisée en utilisant des techniques telles que la courbe de K-distance ou en évaluant les résultats visuellement

# DBSCAN

## Techniques pour choisir les paramètres:

1. **Méthode du k-distance graph:**

    - Calculer la distance au k-ème voisin le plus proche pour chaque point.
    - Tracer la courbe de ces distances en fonction du k.
    - Le "coude" de cette courbe peut indiquer une valeur appropriée pour ε.
2. **Validation croisée:**

    - Diviser les données en plusieurs sous-ensembles.
    - Entraîner le modèle avec différentes combinaisons de ε et MinPts sur les sous-ensembles d'entraînement.
    - Évaluer les performances sur les sous-ensembles de test en utilisant une métrique appropriée (comme le score de silhouette).

# DBSCAN

## Techniques pour choisir les paramètres:

3. **Analyse de sensibilité:**

- Faire varier les valeurs de ε et MinPts sur une grille et observer l'impact sur les résultats.
- Visualiser les clusters obtenus pour différentes combinaisons de paramètres.

4. **Heuristiques basées sur la densité:**
Utiliser des méthodes pour estimer la densité locale des données et ajuster ε en conséquence.

5. **Domaine d'application:** La connaissance du domaine d'application peut guider le choix des paramètres. Par exemple, si l'on s'attend à des clusters de tailles très différentes, il peut être nécessaire d'ajuster MinPts en conséquence.

6. **Itération:** Le choix des paramètres est souvent un processus itératif. Il peut être nécessaire d'affiner les valeurs initiales après une première exploration.

# DBSCAN

## Implémentation en Python avec Scikit-learn de DBSCAN

```python
import numpy as np
from sklearn.cluster import DBSCAN
import matplotlib.pyplot as plt

# Générer des données de démonstration (deux clusters)
X = np.concatenate([
    np.random.randn(50, 2) + [0, 0],
    np.random.randn(50, 2) + [3, 3]
])

# Créer et entraîner le modèle
dbscan = DBSCAN(eps=0.3, min_samples=5)
labels = dbscan.fit_predict(X)

# Visualiser les clusters
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis')
plt.title('Clustering DBSCAN')
plt.show()
```

# DBSCAN

## Implémentation en Python avec Scikit-learn de DBSCAN

### Choix des paramètres

Comme mentionné précédemment, le choix de eps et **min_samples** est crucial. Une méthode courante pour déterminer eps est d'utiliser le graphique de la distance au **k-ème** voisin le plus proche. Le "coude" de ce graphique peut indiquer une valeur appropriée pour eps.

### Autres considérations

- **Données bruitées:** DBSCAN est particulièrement efficace pour gérer les données bruitées. Les points qui ne sont ni des points centraux ni des points de bordure sont considérés comme du bruit et sont assignés à un label unique (souvent -1).
- **Données de grande dimension:** Pour les données de haute dimension, la métrique de distance euclidienne peut ne pas être adaptée. Des métriques telles que la distance de Manhattan ou la distance de Cosinus peuvent être plus appropriées.
- **Visualisation:** Pour des données en dimension supérieure à 3, des techniques de réduction de dimensionnalité (comme t-SNE ou UMAP) peuvent être utilisées pour visualiser les résultats.

# DBSCAN

## Évaluation de la qualité des clusters obtenus par DBSCAN

Voici quelques métriques couramment utilisées pour évaluer la qualité des clusters DBSCAN :

### Métriques internes:

Ces métriques évaluent la qualité des clusters en fonction des données elles-mêmes, sans faire référence à une étiquette de classe connue.

- **Score de silhouette:**
  - Mesure la cohésion à l'intérieur des clusters et la séparation entre les clusters.
  - Une valeur proche de 1 indique une bonne classification, tandis qu'une valeur proche de -1 indique une mauvaise classification.
- **Indice de Davies-Bouldin:**
  - Mesure le rapport entre la distance moyenne entre les clusters et la dispersion à l'intérieur des clusters.
  - Une valeur faible indique une bonne séparation entre les clusters

# DBSCAN

## Évaluation de la qualité des clusters obtenus par DBSCAN

### Métriques internes:

- **Indice de Calinski-Harabasz:**
  - ❖ Mesure le rapport entre la dispersion entre les clusters et la dispersion à l'intérieur des clusters.
  - ❖ Une valeur élevée indique une bonne séparation entre les clusters.

### Métriques externes:

Ces métriques comparent les résultats du clustering avec une étiquette de classe connue (si disponible).

**Indice de Rand ajusté:**

-Mesure la similarité entre deux partitions (la partition obtenue par le clustering et la partition de référence).

-Une valeur proche de 1 indique une forte concordance entre les deux partitions.

# DBSCAN

## Évaluation de la qualité des clusters obtenus par DBSCAN

### Métriques externes:

**Indice de Jaccard:**

-Mesure la similarité entre deux ensembles (les éléments d'un cluster dans une partition et les éléments du même cluster dans l'autre partition).

-Une valeur proche de 1 indique une forte similarité.

**Matrice de confusion:**

Permet de visualiser les erreurs de classification et de calculer des métriques comme la précision, le rappel et le F1-score.

# DBSCAN

## Évaluation de la qualité des clusters obtenus par DBSCAN

### Exemple avec Python:

```
from sklearn.metrics import silhouette_score, davies_bouldin_score,
calinski_harabasz_score
from sklearn.metrics.cluster import adjusted_rand_score

# ... (code pour effectuer le clustering DBSCAN)

# Calcul des métriques
silhouette_avg = silhouette_score(X, labels)
db_index = davies_bouldin_score(X, labels)
ch_index = calinski_harabasz_score(X, labels)
ari = adjusted_rand_score(y_true, labels)  # si y_true est disponible

print("Silhouette Coefficient:", silhouette_avg)
```

# DBSCAN

## Évaluation de la qualité des clusters obtenus par DBSCAN

### Choix de la métrique:

Le choix de la métrique dépend de :

- **La nature des données:** Les données sont-elles continues, discrètes, ordinales ?
- **La forme des clusters:** Les clusters sont-ils sphériques, allongés, de densité variable ?
- **L'objectif de l'analyse:** Souhaite-t-on maximiser la cohésion intra-cluster, minimiser la dispersion inter-cluster, ou les deux ?

# DBSCAN

## Avantages de DBSCAN

- **Détection de clusters de formes arbitraires:** DBSCAN peut identifier des clusters de formes non convexes, contrairement à K-means.
- **Gestion du bruit:** L'algorithme identifie les points aberrants et ne les assigne à aucun cluster.
- **Pas besoin de spécifier le nombre de clusters à l'avance:** DBSCAN détermine automatiquement le nombre de clusters en fonction de la densité des données.

## Limites de DBSCAN

- **Sensibilité aux paramètres:** Les performances de DBSCAN dépendent fortement des valeurs de ε et MinPts.
- **Difficulté à gérer des données de densité variable:** Si les données ont des densités très variables, il peut être difficile de trouver des valeurs de paramètres adaptées.

# LE CLUSTERING HIÉRARCHIQUE

# LE CLUSTERING HIÉRARCHIQUE

## Introduction:

Le clustering hiérarchique est une technique d'apprentissage non supervisé qui vise à regrouper des données en une hiérarchie de clusters. Contrairement au clustering k-means qui nécessite de spécifier à l'avance le nombre de clusters, le clustering hiérarchique construit un arbre de clusters, permettant une exploration plus flexible des données.

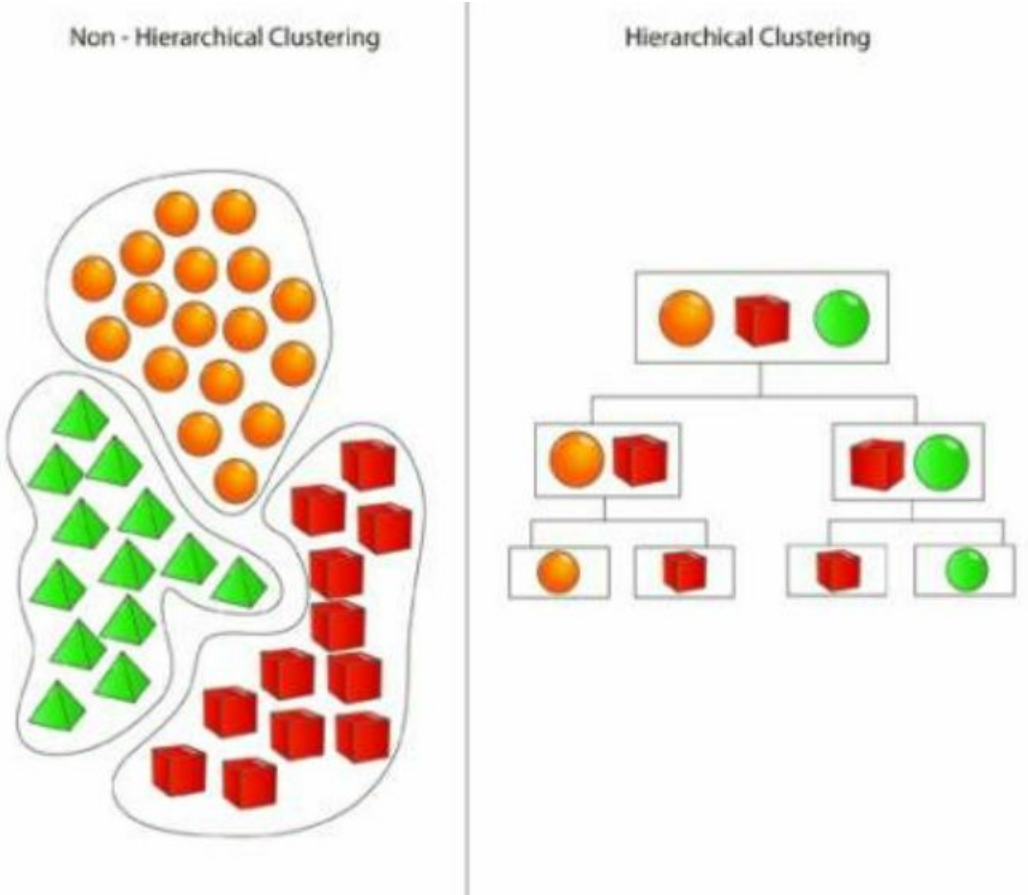
## Principe de fonctionnement:

Le clustering hiérarchique fonctionne de manière agglomérative ou divisive :

- **Agglomérative (**approche ascendante):On commence par considérer chaque point comme un cluster individuel, puis on fusionne itérativement les clusters les plus proches jusqu'à ce qu'il n'en reste qu'un.
- **Divisive (**approche descendante)**:** On commence par considérer toutes les données comme un seul cluster, puis on le divise récursivement en sous-clusters jusqu'à ce que chaque point soit dans un cluster individuel.

# LE CLUSTERING HIÉRARCHIQUE

## Introduction



# LE CLUSTERING HIÉRARCHIQUE

## Méthodes de liaison:

Le choix de la méthode de liaison, qui définit la distance entre deux clusters, est crucial :

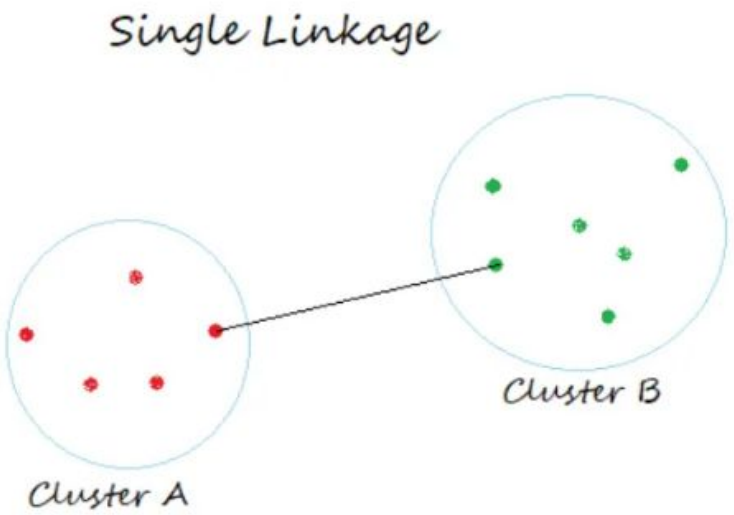
- **Single linkage:** La distance entre deux clusters est définie par la distance minimale entre les points des deux clusters. Cette méthode peut conduire à des chaînes de clusters.
- **Complete linkage:** La distance entre deux clusters est définie par la distance maximale entre les points des deux clusters. Cette méthode tend à produire des clusters plus compacts.
- **Average linkage:** La distance entre deux clusters est définie par la moyenne des distances entre toutes les paires de points des deux clusters.
- **Centroid linkage:** La distance entre deux clusters est définie par la distance entre les centroïdes des deux clusters.
- **Ward's linkage:** Minimise la somme des carrés des erreurs à chaque étape de fusion.

# LE CLUSTERING HIÉRARCHIQUE

## Méthodes de liaison:Liaison Simple

**Principe:** La distance entre deux clusters est définie par la distance minimale entre les points des deux clusters.

**Effet:** Tend à former des chaînes de clusters allongés.

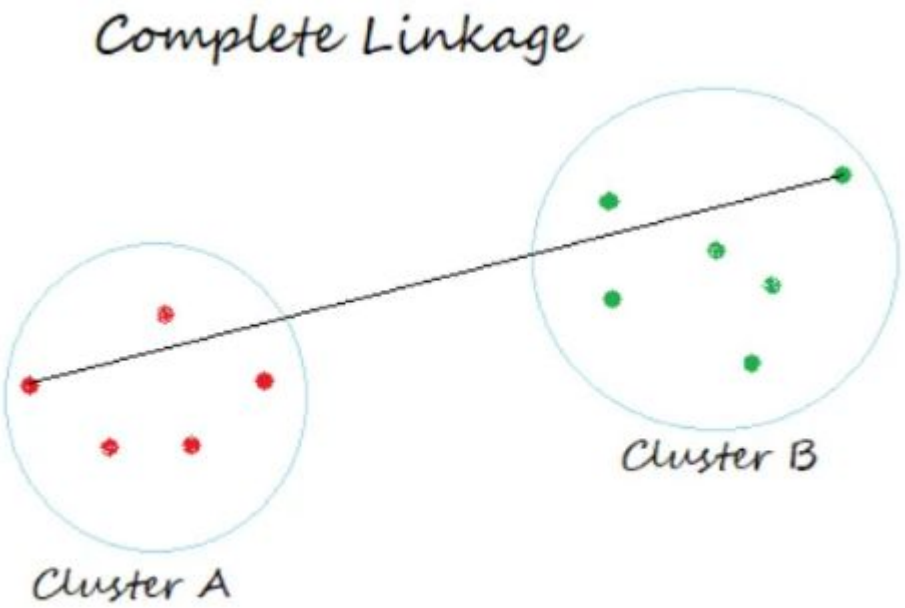


# LE CLUSTERING HIÉRARCHIQUE

## Méthodes de liaison:Liaison Complète

**Principe:** La distance entre deux clusters est définie par la distance maximale entre les points des deux clusters.

**Effet:** Tend à former des clusters compacts et sphériques.

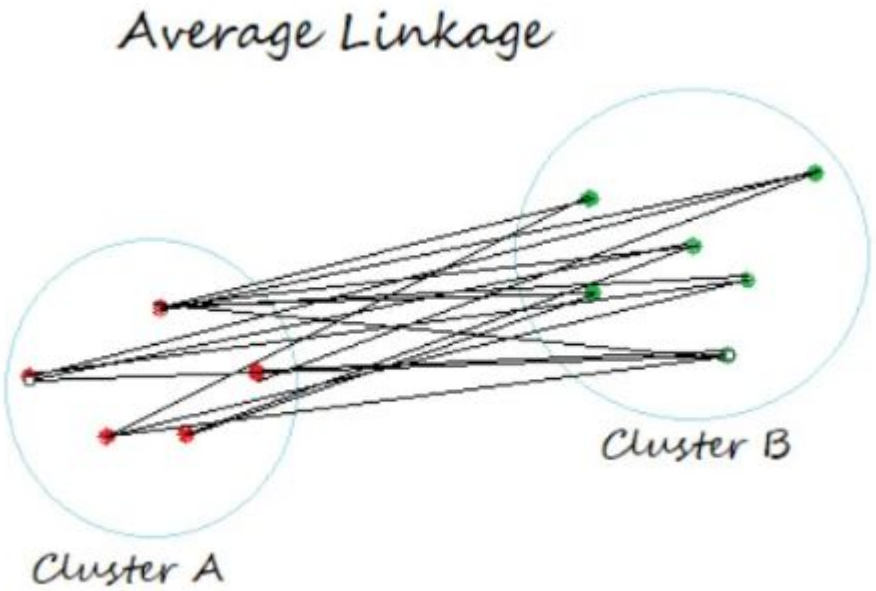


# LE CLUSTERING HIÉRARCHIQUE

## Méthodes de liaison:Liaison Moyenne

**Principe:** La distance entre deux clusters est définie par la distance maximale entre les points des deux clusters.

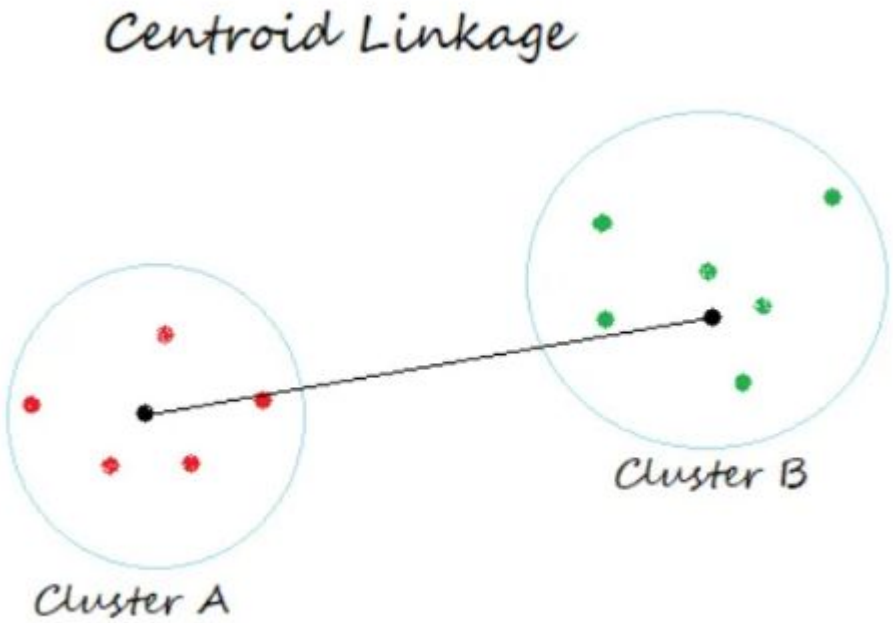
**Effet:** Tend à former des clusters compacts et sphériques.



# LE CLUSTERING HIÉRARCHIQUE

## Méthodes de liaison:Liaison Centroid

La **liaison de centroïde** est une méthode utilisée dans le clustering hiérarchique pour déterminer la distance entre deux clusters. Elle consiste à calculer la distance entre les centroïdes (les points moyens) de ces deux clusters.

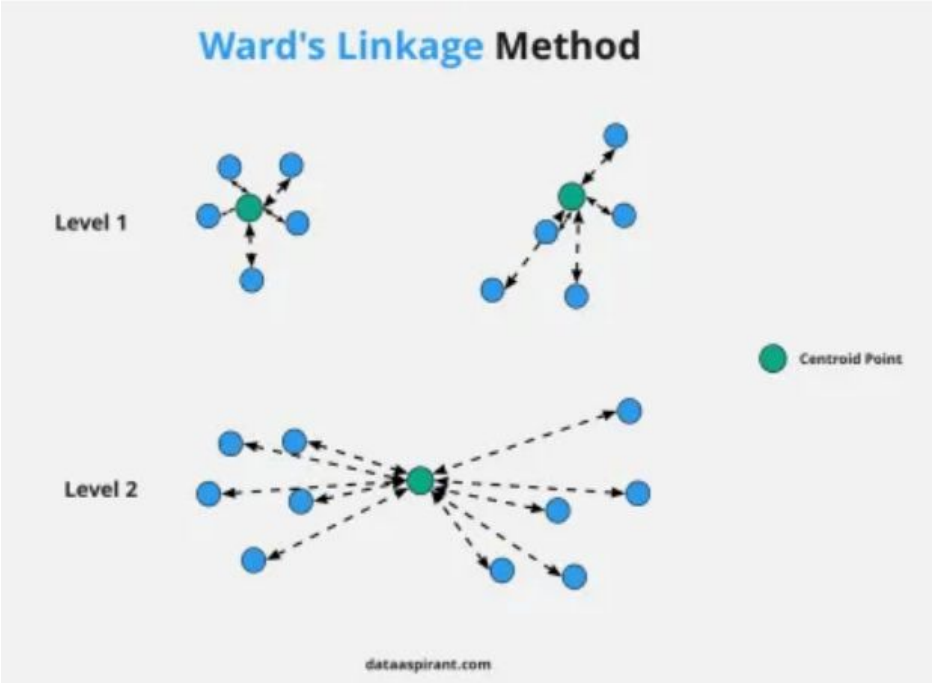


# LE CLUSTERING HIÉRARCHIQUE

## Méthodes de liaison:Liaison de Ward

**Principe:** Minimise la somme des carrés des erreurs à chaque étape de fusion.

**Effet:** Tend à produire des clusters de taille similaire et à minimiser la variance intra-cluster.



# LE CLUSTERING HIÉRARCHIQUE

## Méthodes de liaison

```
# Liste des méthodes de liaison à tester
linkage_methods = ['ward', 'complete', 'average', 'single', 'centroid']
```

Boucle pour tester chaque méthode de liaison
for method in linkage_methods:
    # Clustering hiérarchique
    cluster = AgglomerativeClustering(n_clusters=None, affinity='euclidean', linkage=method)

# LE CLUSTERING HIÉRARCHIQUE

## Représentation : Le dendrogramme

Le résultat d'un clustering hiérarchique est généralement représenté sous forme de dendrogramme. Un dendrogramme est un diagramme en arbre qui montre comment les clusters sont fusionnés à chaque étape. La hauteur d'une fusion représente la distance entre les deux clusters fusionnés.

-C'est tout simplement ***l'arbre qui regroupe les différents clusters.***

-Il n'est pas nécessaire de spécifier un nombre de clusters initial pour lancer l'algorithme.

-Ce nombre de clusters peut se déterminer à posteriori en prenant en compte la profondeur du dendrogramme.

# LE CLUSTERING HIÉRARCHIQUE

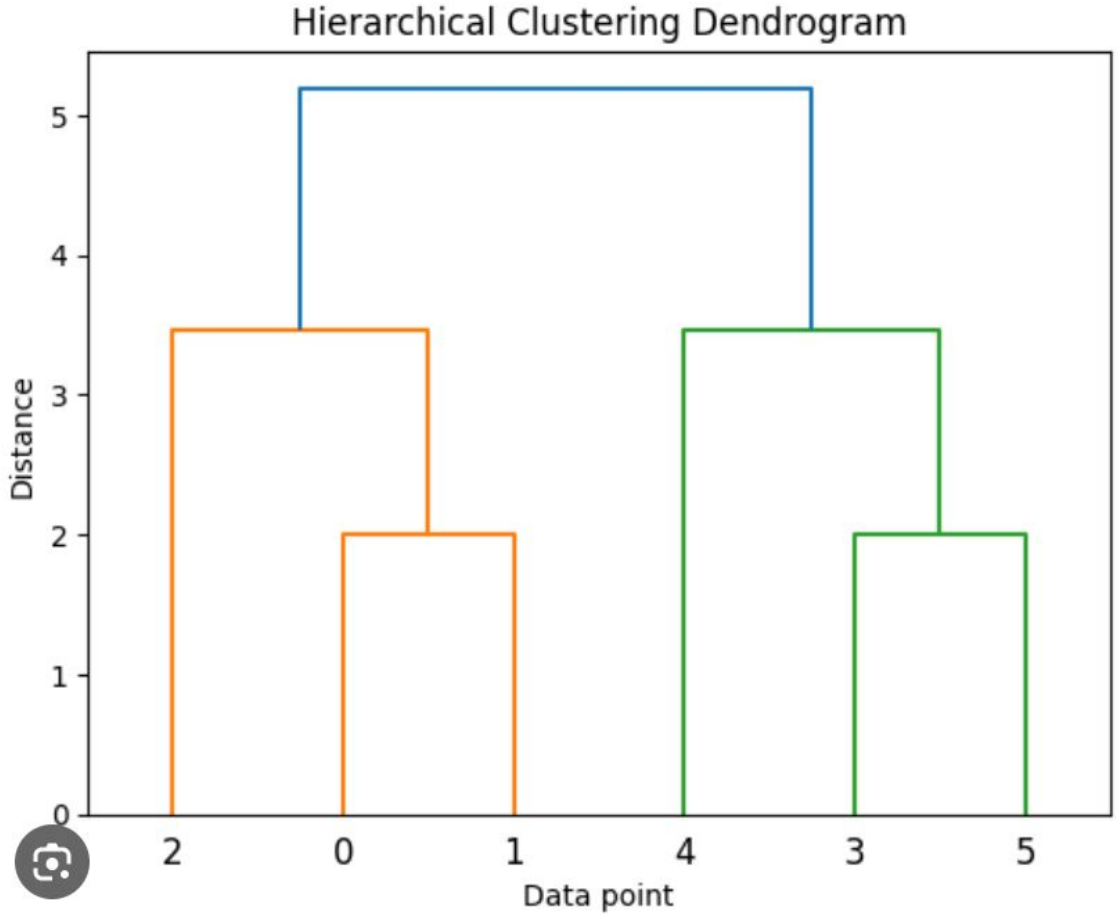
## Structure d'un dendrogramme

- **Axes:** L'axe horizontal représente les observations (ou les clusters), tandis que l'axe vertical représente la distance (ou la similarité) entre les clusters.
- **Branches:** Les lignes verticales représentent les clusters, et leur longueur indique la distance à laquelle les clusters ont été fusionnés.
- **Noeuds:** Les points de fusion des branches correspondent aux moments où deux clusters ont été regroupés en un seul.

## Interprétation d'un dendrogramme

- **Hauteur de fusion:** Plus la hauteur à laquelle deux clusters sont fusionnés est élevée, plus ils sont différents.
- **Nombre de clusters:** En coupant le dendrogramme à un niveau horizontal donné, on obtient une partition des données en un certain nombre de clusters.
- **Hiérarchie des clusters:** Le dendrogramme montre la relation hiérarchique entre les clusters.

# LE CLUSTERING HIÉRARCHIQUE



# LE CLUSTERING HIÉRARCHIQUE

## Implémentation en Python avec Scikit-learn

```
from sklearn.cluster import AgglomerativeClustering
import matplotlib.pyplot as plt
import scipy.cluster.hierarchy as sch

# ... (chargement des données)

# Clustering hiérarchique avec linkage 'ward'
cluster = AgglomerativeClustering(n_clusters=None, affinity='euclidean', linkage='ward')
cluster.fit_predict(X)

# Dendrogramme
plt.figure(figsize=(10, 7))
plt.title("Dendrogramme")
dend = sch.dendrogram(sch.linkage(X, method='ward'))
plt.xlabel("Index")
plt.ylabel("Distance")
plt.show()
```

# LE CLUSTERING HIÉRARCHIQUE

## Implémentation en Python avec Scikit-learn

-`sklearn.cluster`: Pour l'algorithme de clustering hiérarchique.

-`scipy.cluster.hierarchy`: Pour construire le dendrogramme.

-Nous instancions un objet `AgglomerativeClustering` avec :

- `n_clusters=None`: Nous laissons l'algorithme déterminer le nombre optimal de clusters.
- `affinity='euclidean'`: Nous utilisons la distance euclidienne pour mesurer la similarité entre les points.
- `linkage='ward'`: Nous utilisons la méthode de liaison Ward, qui minimise la variance intra-cluster.
- `cluster.fit_predict(X)` applique l'algorithme aux données et retourne les étiquettes de cluster.

-Nous utilisons `sch.dendrogram` pour visualiser la hiérarchie des clusters. La hauteur des lignes verticales représente la distance à laquelle les clusters sont fusionnés.

# LE CLUSTERING HIÉRARCHIQUE

Le dendrogramme nous permet de visualiser comment les clusters sont fusionnés au fur et à mesure que l'algorithme progresse. En coupant le dendrogramme à un niveau horizontal donné, nous obtenons une partition des données en un certain nombre de clusters.

**Choix du nombre de clusters**

Contrairement au k-means, le clustering hiérarchique ne nécessite pas de spécifier a priori le nombre de clusters. Cependant, pour obtenir une partition finale, il faut couper le dendrogramme à un certain niveau. Plusieurs méthodes existent pour déterminer ce niveau :

- **Méthode du coude:** On cherche un "coude" marqué dans le dendrogramme.
- **Indice de silhouette:** On calcule le score de silhouette pour différents nombres de clusters et on choisit le nombre qui maximise ce score.
- **Connaissance du domaine:** Parfois, la connaissance du domaine peut nous guider dans le choix du nombre de clusters.

# LE CLUSTERING HIÉRARCHIQUE

## Applications du Clustering Hiérarchique dans Divers Domaines

### Biologie

- **Phylogénie:** Le clustering hiérarchique est largement utilisé pour reconstruire les arbres phylogénétiques, qui représentent les relations évolutives entre les espèces. En analysant les séquences génétiques, on peut créer des dendrogrammes qui montrent comment les espèces sont apparentées.
- **Expression génique:** Il permet de regrouper des gènes ayant des profils d'expression similaires, ce qui peut aider à identifier des voies métaboliques ou des processus biologiques communs.
- **Classification de protéines:** On peut regrouper des protéines en fonction de leur structure ou de leur fonction, ce qui facilite l'annotation fonctionnelle des nouvelles protéines.

# LE CLUSTERING HIÉRARCHIQUE

## Applications du Clustering Hiérarchique dans Divers Domaines

### Finance

- **Segmentation de clientèle:** Les banques et les sociétés de gestion d'actifs utilisent le clustering hiérarchique pour segmenter leurs clients en fonction de leur profil de risque, de leurs habitudes de consommation ou de leurs démos. Cela permet de proposer des produits et services personnalisés.
- **Analyse de portefeuille:** Le clustering hiérarchique peut être utilisé pour identifier des groupes d'actions ayant des comportements similaires, ce qui aide à construire des portefeuilles diversifiés.
- **Détection de fraudes:** En analysant les transactions financières, on peut identifier des groupes d'activités inhabituelles qui pourraient être liées à des fraudes.

# LE CLUSTERING HIÉRARCHIQUE

## Applications du Clustering Hiérarchique dans Divers Domaines

### Marketing

- **Segmentation de marché:** Le clustering hiérarchique permet de diviser un marché en segments distincts de consommateurs ayant des besoins et des préférences similaires. Cela aide les entreprises à cibler leurs campagnes marketing de manière plus efficace.
- **Analyse de panier:** En analysant les produits achetés ensemble, on peut identifier des associations entre produits et ainsi optimiser les recommandations produits.
- **Analyse de la concurrence:** En comparant les produits ou les marques de différents concurrents, on peut identifier des opportunités de positionnement.

# LE CLUSTERING HIÉRARCHIQUE

## Applications du Clustering Hiérarchique dans Divers Domaines

### Autres domaines

- **Traitement d'images:** Le clustering hiérarchique peut être utilisé pour segmenter des images en régions homogènes, ce qui est utile pour la compression d'images, la reconnaissance de formes ou la segmentation médicale.
- **Sciences sociales:** Il peut être utilisé pour analyser des réseaux sociaux, des textes ou des données sociodémographiques.
- **Géologie:** Le clustering hiérarchique peut aider à identifier des zones géographiques ayant des caractéristiques similaires.

# LE CLUSTERING HIÉRARCHIQUE

**Exemple concret en marketing : Segmentation de clients**

Imaginez une entreprise de vente en ligne de vêtements. En utilisant le clustering hiérarchique, elle pourrait segmenter sa clientèle en fonction de :

- **Les produits achetés:** Les clients qui achètent principalement des vêtements de sport seront dans un cluster différent de ceux qui achètent des vêtements de soirée.
- **Le montant dépensé:** Les clients haut de gamme seront séparés des clients à petit budget.
- **La fréquence d'achat:** Les clients réguliers seront distingués des clients occasionnels.

En ayant une segmentation précise de sa clientèle, l'entreprise pourra personnaliser ses campagnes marketing, proposer des offres ciblées et améliorer l'expérience client.

.

# LE CLUSTERING HIÉRARCHIQUE

## Exemple concret en marketing : Segmentation de clients

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import AgglomerativeClustering
import scipy.cluster.hierarchy as sch

# Simuler des données de clients
# Chaque ligne représente un client, les colonnes représentent les quantités achetées pour chaque catégorie
data = {'Vêtements_sport': [10, 2, 5, 15, 8],
        'Vêtements_soirée': [2, 12, 4, 1, 6],
        'Accessoires': [5, 3, 7, 2, 9]}
df = pd.DataFrame(data)

# Normalisation des données (important pour les algorithmes de distance)
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
data_scaled = scaler.fit_transform(df)

# Clustering hiérarchique avec la méthode de liaison Ward
cluster = AgglomerativeClustering(n_clusters=None, affinity='euclidean', linkage='ward')
cluster.fit_predict(data_scaled)
```

```
# Dendrogramme
plt.figure(figsize=(10, 7))
plt.title("Dendrogramme")
dend = sch.dendrogram(sch.linkage(data_scaled, method='ward'))
plt.xlabel("Index du client")
plt.ylabel("Distance")
plt.show()

# Visualisation des clusters (si les données sont en 2D, sinon utiliser des techniques de réduction de dimension)
plt.scatter(data_scaled[:,0], data_scaled[:,1], c=cluster.labels_, cmap='viridis')
plt.title("Segmentation des clients")
plt.xlabel("Vêtements de sport (normalisés)")
plt.ylabel("Vêtements de soirée (normalisés)")
plt.show()
```

# LE CLUSTERING HIÉRARCHIQUE

## Avantages et limites

- **Avantages:**
  - Ne nécessite pas de spécifier le nombre de clusters à l'avance.
  - Permet de visualiser la hiérarchie des clusters grâce au dendrogramme.
  - Flexible pour différents types de données.
- **Limites:**
  - Peut être coûteux en calcul pour de grands ensembles de données.
  - Le choix de la méthode de liaison peut influencer les résultats.
  - Le choix du niveau de coupe du dendrogramme peut être subjectif.